

CONTENTS IN DETAIL

ACKNOWLEDGMENTS

xxvii

INTRODUCTION

xxix

Who Is This Book For?	xxx
What Can You Expect to Learn?	xxx
Why Python?	xxxi

PART I: BASICS

1

1

GETTING STARTED

3

Setting Up Your Programming Environment	3
Python 2 and Python 3.	4
Running Snippets of Python Code	4
Hello World!	4
Python on Different Operating Systems	5
Python on Linux.	5
Python on OS X	8
Python on Windows	10
Troubleshooting Installation Issues	15
Running Python Programs from a Terminal.	16
On Linux and OS X.	16
On Windows	16
<i>Exercise 1-1: python.org</i>	17
<i>Exercise 1-2: Hello World Typos</i>	17
<i>Exercise 1-3: Infinite Skills</i>	17
Summary	17

2

VARIABLES AND SIMPLE DATA TYPES

19

What Really Happens When You Run <code>hello_world.py</code>	19
Variables	20
Naming and Using Variables	21
Avoiding Name Errors When Using Variables	21
<i>Exercise 2-1: Simple Message</i>	23
<i>Exercise 2-2: Simple Messages</i>	23
Strings	23
Changing Case in a String with Methods.	24
Combining or Concatenating Strings	25
Adding Whitespace to Strings with Tabs or Newlines	26
Stripping Whitespace	26
Avoiding Syntax Errors with Strings	28
Printing in Python 2	29
<i>Exercise 2-3: Personal Message</i>	29
<i>Exercise 2-4: Name Cases</i>	29
<i>Exercise 2-5: Famous Quote</i>	29

<i>Exercise 2-6: Famous Quote 2</i>	29
<i>Exercise 2-7: Stripping Names</i>	29
Numbers	30
Integers	30
Floats	30
Avoiding Type Errors with the <code>str()</code> Function	31
Integers in Python 2	32
<i>Exercise 2-8: Number Eight</i>	33
<i>Exercise 2-9: Favorite Number</i>	33
Comments	33
How Do You Write Comments?	33
What Kind of Comments Should You Write?	33
<i>Exercise 2-10: Adding Comments</i>	34
The Zen of Python	34
<i>Exercise 2-11: Zen of Python</i>	36
Summary	36

3

INTRODUCING LISTS

37

What Is a List?	37
Accessing Elements in a List	38
Index Positions Start at 0, Not 1	39
Using Individual Values from a List	39
<i>Exercise 3-1: Names</i>	40
<i>Exercise 3-2: Greetings</i>	40
<i>Exercise 3-3: Your Own List</i>	40
Changing, Adding, and Removing Elements	40
Modifying Elements in a List	40
Adding Elements to a List	41
Removing Elements from a List	42
<i>Exercise 3-4: Guest List</i>	46
<i>Exercise 3-5: Changing Guest List</i>	46
<i>Exercise 3-6: More Guests</i>	46
<i>Exercise 3-7: Shrinking Guest List</i>	47
Organizing a List	47
Sorting a List Permanently with the <code>sort()</code> Method	47
Sorting a List Temporarily with the <code>sorted()</code> Function	48
Printing a List in Reverse Order	49
Finding the Length of a List	49
<i>Exercise 3-8: Seeing the World</i>	50
<i>Exercise 3-9: Dinner Guests</i>	50
<i>Exercise 3-10: Every Function</i>	50
Avoiding Index Errors When Working with Lists	50
<i>Exercise 3-11: Intentional Error</i>	52
Summary	52

4

WORKING WITH LISTS

53

Looping Through an Entire List	53
A Closer Look at Looping	54
Doing More Work Within a <code>for</code> Loop	55
Doing Something After a <code>for</code> Loop	56

Avoiding Indentation Errors	57
Forgetting to Indent	57
Forgetting to Indent Additional Lines	58
Indenting Unnecessarily	59
Indenting Unnecessarily After the Loop	59
Forgetting the Colon	60
<i>Exercise 4-1: Pizzas</i>	60
<i>Exercise 4-2: Animals</i>	60
Making Numerical Lists	61
Using the range() Function	61
Using range() to Make a List of Numbers	62
Simple Statistics with a List of Numbers	63
List Comprehensions	63
<i>Exercise 4-3: Counting to Twenty</i>	64
<i>Exercise 4-4: One Million</i>	64
<i>Exercise 4-5: Summing a Million</i>	64
<i>Exercise 4-6: Odd Numbers</i>	64
<i>Exercise 4-7: Threes</i>	64
<i>Exercise 4-8: Cubes</i>	64
<i>Exercise 4-9: Cube Comprehension</i>	64
Working with Part of a List	65
Slicing a List	65
Looping Through a Slice	66
Copying a List	67
<i>Exercise 4-10: Slices</i>	69
<i>Exercise 4-11: My Pizzas, Your Pizzas</i>	69
<i>Exercise 4-12: More Loops</i>	69
Tuples	69
Defining a Tuple	69
Looping Through All Values in a Tuple	70
Writing over a Tuple	71
<i>Exercise 4-13: Buffet</i>	71
Styling Your Code	72
The Style Guide	72
Indentation	72
Line Length	73
Blank Lines	73
Other Style Guidelines	73
<i>Exercise 4-14: PEP 8</i>	74
<i>Exercise 4-15: Code Review</i>	74
Summary	74

5

IF STATEMENTS

75

A Simple Example	76
Conditional Tests	76
Checking for Equality	76
Ignoring Case When Checking for Equality	77
Checking for Inequality	78
Numerical Comparisons	78
Checking Multiple Conditions	79
Checking Whether a Value Is in a List	80

Checking Whether a Value Is Not in a List	81
Boolean Expressions	81
<i>Exercise 5-1: Conditional Tests</i>	82
<i>Exercise 5-2: More Conditional Tests</i>	82
if Statements	82
Simple if Statements	82
if-else Statements	83
The if-elif-else Chain	84
Using Multiple elif Blocks	86
Omitting the else Block	86
Testing Multiple Conditions	87
<i>Exercise 5-3: Alien Colors #1</i>	88
<i>Exercise 5-4: Alien Colors #2</i>	88
<i>Exercise 5-5: Alien Colors #3</i>	89
<i>Exercise 5-6: Stages of Life</i>	89
<i>Exercise 5-7: Favorite Fruit</i>	89
Using if Statements with Lists	89
Checking for Special Items	90
Checking That a List Is Not Empty	91
Using Multiple Lists	92
<i>Exercise 5-8: Hello Admin</i>	93
<i>Exercise 5-9: No Users</i>	93
<i>Exercise 5-10: Checking Usernames</i>	93
<i>Exercise 5-11: Ordinal Numbers</i>	93
Styling Your if Statements	94
<i>Exercise 5-12: Styling if statements</i>	94
<i>Exercise 5-13: Your Ideas</i>	94
Summary	94

6 **DICTIONARIES** **95**

A Simple Dictionary	96
Working with Dictionaries	96
Accessing Values in a Dictionary	97
Adding New Key-Value Pairs	97
Starting with an Empty Dictionary	98
Modifying Values in a Dictionary	99
Removing Key-Value Pairs	100
A Dictionary of Similar Objects	100
<i>Exercise 6-1: Person</i>	102
<i>Exercise 6-2: Favorite Numbers</i>	102
<i>Exercise 6-3: Glossary</i>	102
Looping Through a Dictionary	102
Looping Through All Key-Value Pairs	103
Looping Through All the Keys in a Dictionary	104
Looping Through a Dictionary's Keys in Order	106
Looping Through All Values in a Dictionary	107
<i>Exercise 6-4: Glossary 2</i>	108
<i>Exercise 6-5: Rivers</i>	108
<i>Exercise 6-6: Polling</i>	108
Nesting	109
A List of Dictionaries	109
A List in a Dictionary	111

A Dictionary in a Dictionary	113
<i>Exercise 6-7: People</i>	114
<i>Exercise 6-8: Pets</i>	115
<i>Exercise 6-9: Favorite Places</i>	115
<i>Exercise 6-10: Favorite Numbers</i>	115
<i>Exercise 6-11: Cities</i>	115
<i>Exercise 6-12: Extensions</i>	115
Summary	115

7

USER INPUT AND WHILE LOOPS 117

How the input() Function Works	118
Writing Clear Prompts	118
Using int() to Accept Numerical Input	119
The Modulo Operator	120
Accepting Input in Python 2.7	121
<i>Exercise 7-1: Rental Car</i>	121
<i>Exercise 7-2: Restaurant Seating</i>	121
<i>Exercise 7-3: Multiples of Ten</i>	121
Introducing while Loops	122
The while Loop in Action	122
Letting the User Choose When to Quit	122
Using a Flag	124
Using break to Exit a Loop	125
Using continue in a Loop	126
Avoiding Infinite Loops	126
<i>Exercise 7-4: Pizza Toppings</i>	127
<i>Exercise 7-5: Movie Tickets</i>	127
<i>Exercise 7-6: Three Exits</i>	128
<i>Exercise 7-7: Infinity</i>	128
Using a while Loop with Lists and Dictionaries	128
Moving Items from One List to Another	128
Removing All Instances of Specific Values from a List	129
Filling a Dictionary with User Input	130
<i>Exercise 7-8: Deli</i>	131
<i>Exercise 7-9: No Pastrami</i>	131
<i>Exercise 7-10: Dream Vacation</i>	131
Summary	131

8

FUNCTIONS 133

Defining a Function	134
Passing Information to a Function	134
Arguments and Parameters	135
<i>Exercise 8-1: Message</i>	135
<i>Exercise 8-2: Favorite Book</i>	135
Passing Arguments	135
Positional Arguments	136
Keyword Arguments	137
Default Values	138

Equivalent Function Calls	139
Avoiding Argument Errors	140
<i>Exercise 8-3: T-Shirt</i>	141
<i>Exercise 8-4: Large Shirts</i>	141
<i>Exercise 8-5: Cities</i>	141
Return Values.	141
Returning a Simple Value	142
Making an Argument Optional	142
Returning a Dictionary	144
Using a Function with a while Loop	145
<i>Exercise 8-6: City Names</i>	146
<i>Exercise 8-7: Album</i>	146
<i>Exercise 8-8: User Albums</i>	146
Passing a List.	147
Modifying a List in a Function	147
Preventing a Function from Modifying a List	149
<i>Exercise 8-9: Magicians</i>	150
<i>Exercise 8-10: Great Magicians</i>	150
<i>Exercise 8-11: Unchanged Magicians</i>	150
Passing an Arbitrary Number of Arguments.	151
Mixing Positional and Arbitrary Arguments	152
Using Arbitrary Keyword Arguments	152
<i>Exercise 8-12: Sandwiches</i>	154
<i>Exercise 8-13: User Profile</i>	154
<i>Exercise 8-14: Cars</i>	154
Storing Your Functions in Modules	154
Importing an Entire Module	154
Importing Specific Functions	156
Using as to Give a Function an Alias	156
Using as to Give a Module an Alias	157
Importing All Functions in a Module	157
Styling Functions	158
<i>Exercise 8-15: Printing Models</i>	159
<i>Exercise 8-16: Imports</i>	159
<i>Exercise 8-17: Styling Functions</i>	159
Summary	159

9

CLASSES

161

Creating and Using a Class.	162
Creating the Dog Class	162
Making an Instance from a Class	164
<i>Exercise 9-1: Restaurant</i>	166
<i>Exercise 9-2: Three Restaurants</i>	166
<i>Exercise 9-3: Users</i>	166
Working with Classes and Instances.	167
The Car Class.	167
Setting a Default Value for an Attribute	168
Modifying Attribute Values	168
<i>Exercise 9-4: Number Served</i>	171
<i>Exercise 9-5: Login Attempts</i>	171

Inheritance	172
The <code>__init__()</code> Method for a Child Class	172
Inheritance in Python 2.7	173
Defining Attributes and Methods for the Child Class	174
Overriding Methods from the Parent Class	175
Instances as Attributes	175
Modeling Real-World Objects	177
<i>Exercise 9-6: Ice Cream Stand</i>	178
<i>Exercise 9-7: Admin</i>	178
<i>Exercise 9-8: Privileges</i>	178
<i>Exercise 9-9: Battery Upgrade</i>	178
Importing Classes	179
Importing a Single Class	179
Storing Multiple Classes in a Module	180
Importing Multiple Classes from a Module	181
Importing an Entire Module	182
Importing All Classes from a Module	182
Importing a Module into a Module	183
Finding Your Own Workflow	184
<i>Exercise 9-10: Imported Restaurant</i>	184
<i>Exercise 9-11: Imported Admin</i>	184
<i>Exercise 9-12: Multiple Modules</i>	184
The Python Standard Library	184
<i>Exercise 9-13: OrderedDict Rewrite</i>	186
<i>Exercise 9-14: Dice</i>	186
<i>Exercise 9-15: Python Module of the Week</i>	186
Styling Classes	186
Summary	187

10

FILES AND EXCEPTIONS

189

Reading from a File	190
Reading an Entire File	190
File Paths	191
Reading Line by Line	193
Making a List of Lines from a File	194
Working with a File's Contents	194
Large Files: One Million Digits	195
Is Your Birthday Contained in Pi?	196
<i>Exercise 10-1: Learning Python</i>	197
<i>Exercise 10-2: Learning C</i>	197
Writing to a File	197
Writing to an Empty File	197
Writing Multiple Lines	198
Appending to a File	199
<i>Exercise 10-3: Guest</i>	199
<i>Exercise 10-4: Guest Book</i>	199
<i>Exercise 10-5: Programming Poll</i>	199
Exceptions	200
Handling the <code>ZeroDivisionError</code> Exception	200
Using <code>try-except</code> Blocks	200
Using Exceptions to Prevent Crashes	201

The else Block	202
Handling the FileNotFoundError Exception	203
Analyzing Text	204
Working with Multiple Files	205
Failing Silently	206
Deciding Which Errors to Report	207
<i>Exercise 10-6: Addition</i>	207
<i>Exercise 10-7: Addition Calculator</i>	208
<i>Exercise 10-8: Cats and Dogs</i>	208
<i>Exercise 10-9: Silent Cats and Dogs</i>	208
<i>Exercise 10-10: Common Words</i>	208
Storing Data	208
Using json.dump() and json.load()	209
Saving and Reading User-Generated Data	210
Refactoring	212
<i>Exercise 10-11: Favorite Number</i>	214
<i>Exercise 10-12: Favorite Number Remembered</i>	214
<i>Exercise 10-13: Verify User</i>	214
Summary	214

11 TESTING YOUR CODE 215

Testing a Function	216
Unit Tests and Test Cases	217
A Passing Test	217
A Failing Test	218
Responding to a Failed Test	219
Adding New Tests	221
<i>Exercise 11-1: City, Country</i>	222
<i>Exercise 11-2: Population</i>	222
Testing a Class	222
A Variety of Assert Methods	222
A Class to Test	223
Testing the AnonymousSurvey Class	225
The setUp() Method	227
<i>Exercise 11-3: Employee</i>	228
Summary	228

PART II: PROJECTS 231

PROJECT 1: ALIEN INVASION

12 A SHIP THAT FIRES BULLETS 235

Planning Your Project	236
Installing Pygame	236
Installing Python Packages with pip	237
Installing Pygame on Linux	238

Installing Pygame on OS X	239
Installing Pygame on Windows	240
Starting the Game Project	240
Creating a Pygame Window and Responding to User Input	241
Setting the Background Color	242
Creating a Settings Class	243
Adding the Ship Image	244
Creating the Ship Class	245
Drawing the Ship to the Screen	246
Refactoring: the game_functions Module	247
The check_events() Function	247
The update_screen() Function	248
<i>Exercise 12-1: Blue Sky</i>	249
<i>Exercise 12-2: Game Character</i>	249
Piloting the Ship	249
Responding to a Keypress	249
Allowing Continuous Movement	250
Moving Both Left and Right	252
Adjusting the Ship's Speed	253
Limiting the Ship's Range	255
Refactoring check_events()	255
A Quick Recap	256
alien_invasion.py	256
settings.py	256
game_functions.py	256
ship.py	257
<i>Exercise 12-3: Rocket</i>	257
<i>Exercise 12-4: Keys</i>	257
Shooting Bullets	257
Adding the Bullet Settings	257
Creating the Bullet Class	258
Storing Bullets in a Group	259
Firing Bullets	260
Deleting Old Bullets	261
Limiting the Number of Bullets	262
Creating the update_bullets() Function	263
Creating the fire_bullet() Function	264
<i>Exercise 12-5: Sideways Shooter</i>	264
Summary	264

13 ALIENS!

265

Reviewing Your Project	266
Creating the First Alien	266
Creating the Alien Class	267
Creating an Instance of the Alien	268
Making the Alien Appear Onscreen	268
Building the Alien Fleet	269
Determining How Many Aliens Fit in a Row	269
Creating Rows of Aliens	270
Creating the Fleet	271

Refactoring create_fleet()	273
Adding Rows	273
Exercise 13-1: Stars	276
Exercise 13-2: Better Stars	276
Making the Fleet Move	276
Moving the Aliens Right	276
Creating Settings for Fleet Direction	277
Checking to See Whether an Alien Has Hit the Edge	278
Dropping the Fleet and Changing Direction	278
Exercise 13-3: Raindrops	279
Exercise 13-4: Steady Rain	279
Shooting Aliens	280
Detecting Bullet Collisions	280
Making Larger Bullets for Testing	281
Repopulating the Fleet	282
Speeding Up the Bullets	283
Refactoring update_bullets()	283
Exercise 13-5: Catch	284
Ending the Game	284
Detecting Alien-Ship Collisions	284
Responding to Alien-Ship Collisions	285
Aliens that Reach the Bottom of the Screen	288
Game Over!	288
Identifying When Parts of the Game Should Run	289
Exercise 13-6: Game Over	290
Summary	290

14

SCORING

291

Adding the Play Button	292
Creating a Button Class	292
Drawing the Button to the Screen	294
Starting the Game	295
Resetting the Game	296
Deactivating the Play Button	297
Hiding the Mouse Cursor	298
Exercise 14-1: Press P to Play	298
Exercise 14-2: Target Practice	298
Leveling Up	299
Modifying the Speed Settings	299
Resetting the Speed	300
Exercise 14-3: Challenging Target Practice	301
Scoring	301
Displaying the Score	301
Making a Scoreboard	303
Updating the Score as Aliens Are Shot Down	304
Making Sure to Score All Hits	305
Increasing Point Values	306
Rounding the Score	307
High Scores	308

Displaying the Level	310
Displaying the Number of Ships	313
Exercise 14-4: All-Time High Score	317
Exercise 14-5: Refactoring	317
Exercise 14-6: Expanding Alien Invasion	317
Summary	317

PROJECT 2: DATA VISUALIZATION

15	
GENERATING DATA	321
Installing matplotlib	322
On Linux	322
On OS X	322
On Windows	323
Testing matplotlib	323
The matplotlib Gallery	323
Plotting a Simple Line Graph	324
Changing the Label Type and Graph Thickness	324
Correcting the Plot.	326
Plotting and Styling Individual Points with scatter().	326
Plotting a Series of Points with scatter().	328
Calculating Data Automatically.	328
Removing Outlines from Data Points	329
Defining Custom Colors	330
Using a Colormap.	330
Saving Your Plots Automatically	331
Exercise 15-1: Cubes.	331
Exercise 15-2: Colored Cubes	331
Random Walks	331
Creating the RandomWalk() Class.	332
Choosing Directions	332
Plotting the Random Walk	333
Generating Multiple Random Walks	334
Styling the Walk	335
Coloring the Points	335
Plotting the Starting and Ending Points.	336
Cleaning Up the Axes	337
Adding Plot Points.	337
Altering the Size to Fill the Screen	338
Exercise 15-3: Molecular Motion	339
Exercise 15-4: Modified Random Walks	339
Exercise 15-5: Refactoring	339
Rolling Dice with Pygal	339
Installing Pygal	340
The Pygal Gallery	340
Creating the Die Class.	340
Rolling the Die	341
Analyzing the Results.	341

Making a Histogram	342
Rolling Two Dice	343
Rolling Dice of Different Sizes	345
<i>Exercise 15-6: Automatic Labels</i>	346
<i>Exercise 15-7: Two D8s</i>	346
<i>Exercise 15-8: Three Dice</i>	346
<i>Exercise 15-9: Multiplication</i>	346
<i>Exercise 15-10: Practicing with Both Libraries</i>	346
Summary	347

16

DOWNLOADING DATA

349

The CSV File Format	350
Parsing the CSV File Headers	350
Printing the Headers and Their Positions	351
Extracting and Reading Data	352
Plotting Data in a Temperature Chart	353
The datetime Module	354
Plotting Dates	355
Plotting a Longer Timeframe	356
Plotting a Second Data Series	357
Shading an Area in the Chart	358
Error-Checking	359
<i>Exercise 16-1: San Francisco</i>	362
<i>Exercise 16-2: Sitka-Death Valley Comparison</i>	362
<i>Exercise 16-3: Rainfall</i>	362
<i>Exercise 16-4: Explore</i>	362
Mapping Global Data Sets: JSON Format	362
Downloading World Population Data	362
Extracting Relevant Data	363
Converting Strings into Numerical Values	364
Obtaining Two-Digit Country Codes	365
Building a World Map	367
Plotting Numerical Data on a World Map	368
Plotting a Complete Population Map	369
Grouping Countries by Population	371
Styling World Maps in Pygal	372
Lightening the Color Theme	374
<i>Exercise 16-5: All Countries</i>	375
<i>Exercise 16-6: Gross Domestic Product</i>	375
<i>Exercise 16-7: Choose Your Own Data</i>	375
<i>Exercise 16-8: Testing the country_codes Module</i>	375
Summary	375

17

WORKING WITH APIS

377

Using a Web API	378
Git and GitHub	378
Requesting Data Using an API Call	378
Installing Requests	379

Processing an API Response	379
Working with the Response Dictionary	380
Summarizing the Top Repositories	382
Monitoring API Rate Limits	383
Visualizing Repositories Using Pygal	384
Refining Pygal Charts	386
Adding Custom Tooltips	387
Plotting the Data	388
Adding Clickable Links to Our Graph	390
The Hacker News API	390
Exercise 17-1: Other Languages	393
Exercise 17-2: Active Discussions	393
Exercise 17-3: Testing python_repos.py	393
Summary	393

PROJECT 3: WEB APPLICATIONS

18

GETTING STARTED WITH DJANGO 397

Setting Up a Project	398
Writing a Spec	398
Creating a Virtual Environment	398
Installing virtualenv	399
Activating the Virtual Environment	399
Installing Django	400
Creating a Project in Django	400
Creating the Database	401
Viewing the Project	401
Exercise 18-1: New Projects	402
Starting an App	403
Defining Models	403
Activating Models	404
The Django Admin Site	406
Defining the Entry Model	408
Migrating the Entry Model	409
Registering Entry with the Admin Site	409
The Django Shell	410
Exercise 18-2: Short Entries	412
Exercise 18-3: The Django API	412
Exercise 18-4: Pizzeria	412
Making Pages: The Learning Log Home Page	412
Mapping a URL	413
Writing a View	414
Writing a Template	415
Exercise 18-5: Meal Planner	416
Exercise 18-6: Pizzeria Home Page	416
Building Additional Pages	416
Template Inheritance	416
The Topics Page	418
Individual Topic Pages	421

<i>Exercise 18-7: Template Documentation</i>	424
<i>Exercise 18-8: Pizzeria Pages</i>	424
Summary	425

19 USER ACCOUNTS 427

Allowing Users to Enter Data	428
Adding New Topics	428
Adding New Entries	432
Editing Entries	435
<i>Exercise 19-1: Blog</i>	438
Setting Up User Accounts	439
The users App	439
The Login Page	440
Logging Out	442
The Registration Page	443
<i>Exercise 19-2: Blog Accounts</i>	446
Allowing Users to Own Their Data	446
Restricting Access with @login_required	447
Connecting Data to Certain Users	448
Restricting Topics Access to Appropriate Users	451
Protecting a User's Topics	451
Protecting the edit_entry Page	452
Associating New Topics with the Current User	453
<i>Exercise 19-3: Refactoring</i>	454
<i>Exercise 19-4: Protecting new_entry</i>	454
<i>Exercise 19-5: Protected Blog</i>	454
Summary	454

20 STYLING AND DEPLOYING AN APP 455

Styling Learning Log	456
The django-bootstrap3 App	456
Using Bootstrap to Style Learning Log	457
Modifying base.html	458
Styling the Home Page Using a Jumbotron	461
Styling the Login Page	461
Styling the new_topic Page	463
Styling the Topics Page	463
Styling the Entries on the Topic Page	464
<i>Exercise 20-1: Other Forms</i>	466
<i>Exercise 20-2: Stylish Blog</i>	466
Deploying Learning Log	466
Making a Heroku Account	466
Installing the Heroku Toolbelt	466
Installing Required Packages	466
Creating a Packages List with a requirements.txt File	467
Specifying the Python Runtime	468
Modifying settings.py for Heroku	468
Making a Procfile to Start Processes	469
Modifying wsgi.py for Heroku	470

Making a Directory for Static Files	470
Using the unicorn Server Locally	470
Using Git to Track the Project's Files	471
Pushing to Heroku	473
Setting Up the Database on Heroku	474
Refining the Heroku Deployment	475
Securing the Live Project	476
Committing and Pushing Changes	477
Creating Custom Error Pages	478
Ongoing Development	480
The SECRET_KEY Setting	481
Deleting a Project on Heroku	481
<i>Exercise 20-3: Live Blog</i>	<i>482</i>
<i>Exercise 20-4: More 404s</i>	<i>482</i>
<i>Exercise 20-5: Extended Learning Log</i>	<i>482</i>
Summary	482

AFTERWORD 483

A INSTALLING PYTHON 485

Python on Linux	485
Finding the Installed Version	486
Installing Python 3 on Linux	486
Python on OS X	486
Finding the Installed Version	486
Using Homebrew to Install Python 3	487
Python on Windows	488
Installing Python 3 on Windows	488
Finding the Python Interpreter	488
Adding Python to Your Path Variable	489
Python Keywords and Built-in Functions	489
Python Keywords	489
Python Built-in Functions	490

B TEXT EDITORS 491

Geany	492
Installing Geany on Linux	492
Installing Geany on Windows	492
Running Python Programs in Geany	493
Customizing Geany Settings	493
Sublime Text	494
Installing Sublime Text on OS X	494
Installing Sublime Text on Linux	494
Installing Sublime Text on Windows	495
Running Python Programs in Sublime Text	495
Configuring Sublime Text	495
Customizing Sublime Text Settings	496

IDLE	496
Installing IDLE on Linux.	496
Installing IDLE on OS X	496
Installing IDLE on Windows	497
Customizing IDLE Settings	497
Emacs and vim	497

C

GETTING HELP 499

First Steps	499
Try It Again	500
Take a Break	500
Refer to This Book's Resources	500
Searching Online	501
Stack Overflow	501
The Official Python Documentation	501
Official Library Documentation	502
r/learnpython	502
Blog Posts	502
IRC (Internet Relay Chat)	502
Make an IRC Account	502
Channels to Join	503
IRC Culture	503

D

USING GIT FOR VERSION CONTROL 505

Installing Git	506
Installing Git on Linux	506
Installing Git on OS X	506
Installing Git on Windows	506
Configuring Git.	506
Making a Project	507
Ignoring Files.	507
Initializing a Repository.	507
Checking the Status	508
Adding Files to the Repository	508
Making a Commit	509
Checking the Log	509
The Second Commit	510
Reverting a Change	511
Checking Out Previous Commits	512
Deleting the Repository	513

INDEX 515

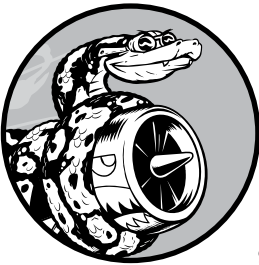
ACKNOWLEDGMENTS

This book would not have been possible without the wonderful and extremely professional staff at No Starch Press. Bill Pollock invited me to write an introductory book, and I deeply appreciate that original offer. Tyler Ortman helped shape my thinking in the early stages of drafting. Liz Chadwick's and Leslie Shen's initial feedback on each chapter was invaluable, and Anne Marie Walker helped to clarify many parts of the book. Riley Hoffman answered every question I had about the process of assembling a complete book and patiently turned my work into a beautiful finished product.

I'd like to thank Kenneth Love, the technical reviewer for *Python Crash Course*. I met Kenneth at PyCon one year, and his enthusiasm for the language and the Python community has been a constant source of professional inspiration ever since. Kenneth went beyond simple fact-checking and reviewed the book with the goal of helping beginning programmers develop a solid understanding of the Python language and programming in general. That said, any inaccuracies that remain are completely my own.

I'd like to thank my father for introducing me to programming at a young age and for not being afraid that I'd break his equipment. I'd like to thank my wife, Erin, for supporting and encouraging me through the writing of this book, and I'd like to thank my son, Ever, whose curiosity inspires me every single day.

INTRODUCTION



Every programmer has a story about how they learned to write their first program. I started learning as a child when my father was working for Digital Equipment Corporation, one of the pioneering companies of the modern computing era. I wrote my first program on a kit computer my dad had assembled in our basement. The computer consisted of nothing more than a bare motherboard connected to a keyboard without a case, and it had a bare cathode ray tube for a monitor. My initial program was a simple number guessing game, which looked something like this:

```
I'm thinking of a number! Try to guess the number I'm thinking of: 25
Too low! Guess again: 50
Too high! Guess again: 42
That's it! Would you like to play again? (yes/no) no
Thanks for playing!
```

I'll always remember how satisfied I felt watching my family play a game that I created and that worked as I intended it to.

That early experience had a lasting impact. There is real satisfaction in building something with a purpose, something that solves a problem. The software I write now meets a more significant need than my childhood efforts, but the sense of satisfaction I get from creating a program that works is still largely the same.

Who Is This Book For?

The goal of this book is to bring you up to speed with Python as quickly as possible so you can build programs that work—games, data visualizations, and web applications—while developing a foundation in programming that will serve you well for the rest of your life. *Python Crash Course* is written for people of any age who have never programmed in Python before or have never programmed at all. If you want to learn the basics of programming quickly so you can focus on interesting projects, and you like to test your understanding of new concepts by solving meaningful problems, this book is for you. *Python Crash Course* is also perfect for middle school and high school teachers who want to offer their students a project-based introduction to programming.

What Can You Expect to Learn?

The purpose of this book is to make you a good programmer in general and a good Python programmer in particular. You'll learn efficiently and adopt good habits as I provide you with a solid foundation in general programming concepts. After working your way through *Python Crash Course*, you should be ready to move on to more advanced Python techniques, and your next programming language will be even easier to grasp.

In the first part of this book you'll learn basic programming concepts you need to know to write Python programs. These concepts are the same as those you'd learn when starting out in almost any programming language. You'll learn about different kinds of data and the ways you can store data in lists and dictionaries within your programs. You'll learn to build collections of data and work through those collections in efficient ways. You'll learn to use `while` and `if` loops to test for certain conditions so you can run specific sections of code while those conditions are true and run other sections when they're not—a technique that greatly helps to automate processes.

You'll learn to accept input from users to make your programs interactive and to keep your programs running as long as the user is active. You'll explore how to write functions to make parts of your program reusable, so you only have to write blocks of code that perform certain

actions once, which you can then use as many times as you like. You'll then extend this concept to more complicated behavior with classes, making fairly simple programs respond to a variety of situations. You'll learn to write programs that handle common errors gracefully. After working through each of these basic concepts, you'll write a few short programs that solve some well-defined problems. Finally, you'll take your first step toward intermediate programming by learning how to write tests for your code so you can develop your programs further without worrying about introducing bugs. All the information in Part I will prepare you for taking on larger, more complex projects.

In Part II you'll apply what you learned in Part I to three projects. You can do any or all of these projects in whichever order works best for you. In the first project (Chapters 12–14) you'll create a Space Invaders–style shooting game called Alien Invasion, which consists of levels of increasing difficulty. After you've completed this project, you should be well on your way to being able to develop your own 2D games.

The second project (Chapters 15–17) introduces you to data visualization. Data scientists aim to make sense of the vast amount of information available to them through a variety of visualization techniques. You'll work with data sets that you generate through code, data sets downloaded from online sources, and data sets your programs download automatically. After you've completed this project, you'll be able to write programs that sift through large data sets and make visual representations of that stored information.

In the third project (Chapters 18–20) you'll build a small web application called Learning Log. This project allows you to keep a journal of ideas and concepts you've learned about a specific topic. You'll be able to keep separate logs for different topics and allow others to create an account and start their own journals. You'll also learn how to deploy your project so anyone can access it online from anywhere.

Why Python?

Every year I consider whether to continue using Python or whether to move on to a different language—perhaps one that's newer to the programming world. But I continue to focus on Python for many reasons. Python is an incredibly efficient language: your programs will do more in fewer lines of code than many other languages would require. Python's syntax will also help you write “clean” code. Your code will be easy to read, easy to debug, and easy to extend and build upon compared to other languages.

People use Python for many purposes: to make games, build web applications, solve business problems, and develop internal tools at all kinds of interesting companies. Python is also used heavily in scientific fields for academic research and applied work.

One of the most important reasons I continue to use Python is because of the Python community, which includes an incredibly diverse and welcoming group of people. Community is essential to programmers because programming isn't a solitary pursuit. Most of us, even the most experienced programmers, need to ask advice from others who have already solved similar problems. Having a well-connected and supportive community is critical in helping you solve problems, and the Python community is fully supportive of people like you who are learning Python as your first programming language.

Python is a great language to learn, so let's get started!

PART I

BASICS

Part I of this book teaches you the basic concepts you'll need to write Python programs. Many of these concepts are common to all programming languages, so they'll be useful throughout your life as a programmer.

In **Chapter 1** you'll install Python on your computer and run your first program, which prints the message *Hello world!* to the screen.

In **Chapter 2** you'll learn to store information in variables and work with text and numerical values.

Chapters 3 and **4** introduce lists. Lists can store as much information as you want in one variable, allowing you to work with that data efficiently. You'll be able to work with hundreds, thousands, and even millions of values in just a few lines of code.

In **Chapter 5** you'll use if statements to write code that responds one way if certain conditions are true, and responds in a different way if those conditions are not true.

Chapter 6 shows you how to use Python's dictionaries, which let you make connections between different pieces of information. Like lists, dictionaries can contain as much information as you need to store.

In **Chapter 7** you'll learn how to accept input from users to make your programs interactive. You'll also learn about while loops, which run blocks of code repeatedly as long as certain conditions remain true.

In **Chapter 8** you'll write functions, which are named blocks of code that perform a specific task and can be run whenever you need them.

Chapter 9 introduces classes, which allow you to model real-world objects, such as dogs, cats, people, cars, rockets, and much more, so your code can represent anything real or abstract.

Chapter 10 shows you how to work with files and handle errors so your programs won't crash unexpectedly. You'll store data before your program closes, and read the data back in when the program runs again. You'll learn about Python's exceptions, which allow you to anticipate errors, and make your programs handle those errors gracefully.

In **Chapter 11** you'll learn to write tests for your code to check that your programs work the way you intend them to. As a result, you'll be able to expand your programs without worrying about introducing new bugs. Testing your code is one of the first skills that will help you transition from beginner to intermediate programmer.

1

GETTING STARTED



In this chapter you'll run your first Python program, *hello_world.py*. First, you'll need to check whether Python is installed on your computer; if it isn't, you'll install it. You'll also install a text editor to work with your Python programs. Text editors recognize Python code and highlight sections as you write, making it easy to understand the structure of your code.

Setting Up Your Programming Environment

Python differs slightly on different operating systems, so you'll need to keep a few considerations in mind. Here, we'll look at the two major versions of Python currently in use and outline the steps to set up Python on your system.

Python 2 and Python 3

Today, two versions of Python are available: Python 2 and the newer Python 3. Every programming language evolves as new ideas and technologies emerge, and the developers of Python have continually made the language more versatile and powerful. Most changes are incremental and hardly noticeable, but in some cases code written for Python 2 may not run properly on systems with Python 3 installed. Throughout this book I'll point out areas of significant difference between Python 2 and Python 3, so whichever version you use, you'll be able to follow the instructions.

If both versions are installed on your system or if you need to install Python, use Python 3. If Python 2 is the only version on your system and you'd rather jump into writing code instead of installing Python, you can start with Python 2. But the sooner you upgrade to using Python 3 the better, so you'll be working with the most recent version.

Running Snippets of Python Code

Python comes with an interpreter that runs in a terminal window, allowing you to try bits of Python without having to save and run an entire program.

Throughout this book, you'll see snippets that look like this:

```
❶ >>> print("Hello Python interpreter!")
Hello Python interpreter!
```

The text in bold is what you'll type in and then execute by pressing ENTER. Most of the examples in the book are small, self-contained programs that you'll run from your editor, because that's how you'll write most of your code. But sometimes basic concepts will be shown in a series of snippets run through a Python terminal session to demonstrate isolated concepts more efficiently. Any time you see the three angle brackets in a code listing ❶, you're looking at the output of a terminal session. We'll try coding in the interpreter for your system in a moment.

Hello World!

A long-held belief in the programming world has been that printing a *Hello world!* message to the screen as your first program in a new language will bring you luck.

In Python, you can write the *Hello World* program in one line:

```
print("Hello world!")
```

Such a simple program serves a very real purpose. If it runs correctly on your system, any Python program you write should work as well. We'll look at writing this program on your particular system in just a moment.

Python on Different Operating Systems

Python is a cross-platform programming language, which means it runs on all the major operating systems. Any Python program you write should run on any modern computer that has Python installed. However, the methods for setting up Python on different operating systems vary slightly.

In this section you'll learn how to set up Python and run the *Hello World* program on your own system. You'll first check whether Python is installed on your system and install it if it's not. Then you'll install a simple text editor and save an empty Python file called *hello_world.py*. Finally, you'll run the *Hello World* program and troubleshoot anything that didn't work. I'll walk you through this process for each operating system, so you'll have a beginner-friendly Python programming environment.

Python on Linux

Linux systems are designed for programming, so Python is already installed on most Linux computers. The people who write and maintain Linux expect you to do your own programming at some point and encourage you to do so. For this reason there's very little you have to install and very few settings you have to change to start programming.

Checking Your Version of Python

Open a terminal window by running the Terminal application on your system (in Ubuntu, you can press CTRL-ALT-T). To find out whether Python is installed, enter **python** with a lowercase *p*. You should see output telling you which version of Python is installed and a `>>>` prompt where you can start entering Python commands, like this:

```
$ python
Python 2.7.6 (default, Mar 22 2014, 22:59:38)
[GCC 4.8.2] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

This output tells you that Python 2.7.6 is currently the default version of Python installed on this computer. When you've seen this output, press CTRL-D or enter `exit()` to leave the Python prompt and return to a terminal prompt.

To check for Python 3, you might have to specify that version; so even if the output displayed Python 2.7 as the default version, try the command `python3`:

```
$ python3
Python 3.5.0 (default, Sep 17 2015, 13:05:18)
[GCC 4.8.4] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

This output means you also have Python 3 installed, so you'll be able to use either version. Whenever you see the `python` command in this book, enter `python3` instead. Most Linux distributions have Python already installed, but if for some reason yours didn't or if your system came with Python 2 and you want to install Python 3, refer to Appendix A.

Installing a Text Editor

Geany is a simple text editor: it's easy to install, will let you run almost all your programs directly from the editor instead of through a terminal, uses syntax highlighting to color your code, and runs your code in a terminal window so you'll get used to using terminals. Appendix B provides information on other text editors, but I recommend using Geany unless you have a good reason to use a different editor.

You can install Geany in one line on most Linux systems:

```
$ sudo apt-get install geany
```

If this doesn't work, see the instructions at <http://geany.org/Download/ThirdPartyPackages/>.

Running the Hello World Program

To start your first program, open Geany. Press the Super key (often called the Windows key) and search for Geany on your system. Make a shortcut by dragging the icon to your taskbar or desktop. Then make a folder somewhere on your system for your projects and call it *python_work*. (It's best to use lowercase letters and underscores for spaces in file and folder names because these are Python naming conventions.) Go back to Geany and save an empty Python file (**File ▶ Save As**) called *hello_world.py* in your *python_work* folder. The extension *.py* tells Geany your file will contain a Python program. It also tells Geany how to run your program and highlight the text in a helpful way.

After you've saved your file, enter the following line:

```
print("Hello Python world!")
```

If multiple versions of Python are installed on your system, you need to make sure Geany is configured to use the correct version. Go to **Build ▶ Set Build Commands**. You should see the words *Compile* and *Execute* with a command next to each. Geany assumes the correct command for each is `python`, but if your system uses the `python3` command, you'll need to change this.

If the command `python3` worked in a terminal session, change the *Compile* and *Execute* commands so Geany will use the Python 3 interpreter. Your *Compile* command should look like this:

```
python3 -m py_compile "%f"
```

You need to type this command exactly as it's shown. Make sure the spaces and capitalization match what is shown here.

Your Execute command should look like this:

```
python3 "%f"
```

Again, make sure the spacing and capitalization match what is shown here. Figure 1-1 shows how these commands should look in Geany's configuration menu.

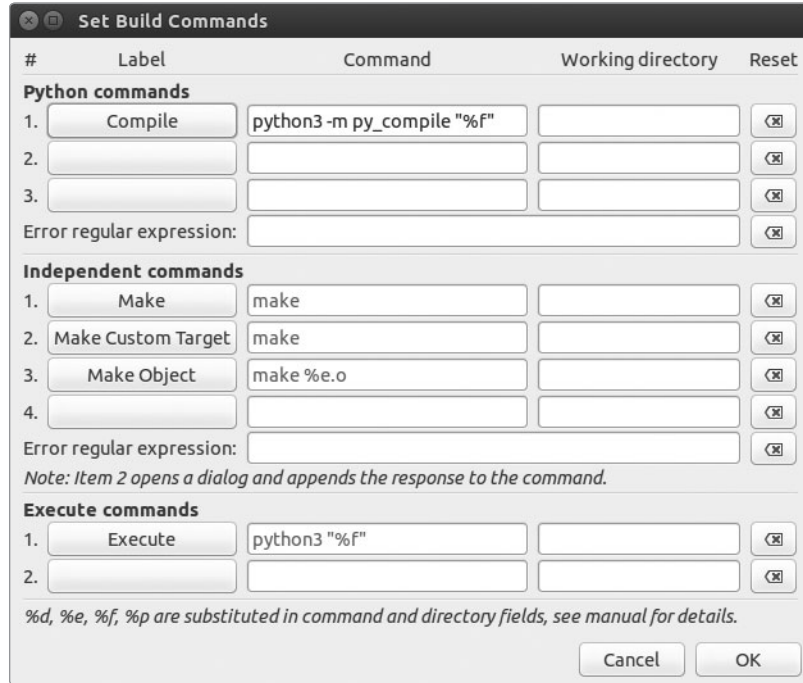


Figure 1-1: Here, Geany is configured to use Python 3 on Linux.

Now run `hello_world.py` by selecting **Build ▶ Execute** in the menu, by clicking the Execute icon (which shows a set of gears), or by pressing F5. A terminal window should pop up with the following output:

```
Hello Python world!
```

```
-----  
(program exited with code: 0)  
Press return to continue
```

If you don't see this, check every character on the line you entered. Did you accidentally capitalize print? Did you forget one or both of the quotation marks or parentheses? Programming languages expect very specific syntax, and if you don't provide that, you'll get errors. If you can't get the program to run, see "Troubleshooting Installation Issues" on page 15.

Running Python in a Terminal Session

You can try running snippets of Python code by opening a terminal and typing `python` or `python3`, as you did when checking your version. Do this again, but this time enter the following line in the terminal session:

```
>>> print("Hello Python interpreter!")
Hello Python interpreter!
>>>
```

You should see your message printed directly in the current terminal window. Remember that you can close the Python interpreter by pressing CTRL-D or by typing the command `exit()`.

Python on OS X

Python is already installed on most OS X systems. Once you know Python is installed, you'll need to install a text editor and make sure it's configured correctly.

Checking Whether Python Is Installed

Open a terminal window by going to **Applications ▸ Utilities ▸ Terminal**. You can also press COMMAND-spacebar, type **terminal**, and then press ENTER. To find out whether Python is installed, enter `python` with a lowercase *p*. You should see output telling you which version of Python is installed on your system and a `>>>` prompt where you can start entering Python commands, like this:

```
$ python
Python 2.7.5 (default, Mar  9 2014, 22:15:05)
[GCC 4.2.1 Compatible Apple LLVM 5.0 (clang-500.0.68)] on darwin
Type "help", "copyright", "credits", or "license" for more information.
>>>
```

This output tells you that Python 2.7.5 is currently the default version installed on this computer. When you've seen this output, press CTRL-D or enter `exit()` to leave the Python prompt and return to a terminal prompt.

To check for Python 3, try the command `python3`. You might get an error message, but if the output shows you have Python 3 installed, you'll be able to use Python 3 without having to install it. If `python3` works on your system, whenever you see the `python` command in this book, make sure you use `python3` instead. If for some reason your system didn't come with Python or if you only have Python 2 and you want to install Python 3 now, see Appendix A.

Running Python in a Terminal Session

You can try running snippets of Python code by opening a terminal and typing `python` or `python3`, as you did when checking your version. Do this again, but this time enter the following line in the terminal session:

```
>>> print("Hello Python interpreter!")
Hello Python interpreter!
>>>
```

You should see your message printed directly in the current terminal window. Remember that you can close the Python interpreter by pressing CTRL-D or by typing the command `exit()`.

Installing a Text Editor

Sublime Text is a simple text editor: it's easy to install on OS X, will let you run almost all of your programs directly from the editor instead of through a terminal, uses syntax highlighting to color your code, and runs your code in a terminal session embedded in the Sublime Text window to make it easy to see the output. Appendix B provides information on other text editors, but I recommend using Sublime Text unless you have a good reason to use a different editor.

You can download an installer for Sublime Text from <http://sublimetext.com/3>. Click the download link and look for an installer for OS X. Sublime Text has a very liberal licensing policy: you can use the editor for free as long as you want, but the author requests that you purchase a license if you like it and want continual use. After the installer has been downloaded, open it and then drag the Sublime Text icon into your *Applications* folder.

Configuring Sublime Text for Python 3

If you use a command other than `python` to start a Python terminal session, you'll need to configure Sublime Text so it knows where to find the correct version of Python on your system. Issue the following command to find out the full path to your Python interpreter:

```
$ type -a python3
python3 is /usr/local/bin/python3
```

Now open Sublime Text, and go to **Tools ▸ Build System ▸ New Build System**, which will open a new configuration file for you. Delete what you see and enter the following:

Python3
.sublime-build

```
{
    "cmd": ["/usr/local/bin/python3", "-u", "$file"],
}
```

This code tells Sublime Text to use your system's `python3` command when running the currently open file. Make sure you use the path you found when issuing the command type `-a python3` in the previous step. Save the file as *Python3.sublime-build* in the default directory that Sublime Text opens when you choose Save.

Running the Hello World Program

To start your first program, launch Sublime Text by opening the *Applications* folder and double-clicking the Sublime Text icon. You can also press `COMMAND-spacebar` and enter **sublime text** in the search bar that pops up.

Make a folder called *python_work* somewhere on your system for your projects. (It's best to use lowercase letters and underscores for spaces in file and folder names, because these are Python naming conventions.) Save an empty Python file (**File ▶ Save As**) called *hello_world.py* in your *python_work* folder. The extension *.py* tells Sublime Text that your file will contain a Python program and tells it how to run your program and highlight the text in a helpful way.

After you've saved your file, enter the following line:

```
print("Hello Python world!")
```

If the command `python` works on your system, you can run your program by selecting **Tools ▶ Build** in the menu or by pressing `CTRL-B`. If you configured Sublime Text to use a command other than `python`, select **Tools ▶ Build System** and then select **Python 3**. This sets Python 3 as the default version of Python, and you'll be able to select **Tools ▶ Build** or just press `COMMAND-B` to run your programs from now on.

A terminal screen should appear at the bottom of the Sublime Text window, showing the following output:

```
Hello Python world!  
[Finished in 0.1s]
```

If you don't see this, check every character on the line you entered. Did you accidentally capitalize `print`? Did you forget one or both of the quotation marks or parentheses? Programming languages expect very specific syntax, and if you don't provide that, you'll get errors. If you can't get the program to run, see "Troubleshooting Installation Issues" on page 15.

Python on Windows

Windows doesn't always come with Python, so you'll probably need to download and install it, and then download and install a text editor.

Installing Python

First, check whether Python is installed on your system. Open a command window by entering **command** into the Start menu or by holding down the SHIFT key while right-clicking on your desktop and selecting **Open command window here**. In the terminal window, enter **python** in lowercase. If you get a Python prompt (**>>>**), Python is installed on your system. However, you'll probably see an error message telling you that python is not a recognized command.

In that case, download a Python installer for Windows. Go to <http://python.org/downloads/>. You should see two buttons, one for downloading Python 3 and one for downloading Python 2. Click the Python 3 button, which should automatically start downloading the correct installer for your system. After you've downloaded the file, run the installer. Make sure you check the option Add Python to PATH, which will make it easier to configure your system correctly. Figure 1-2 shows this option checked.

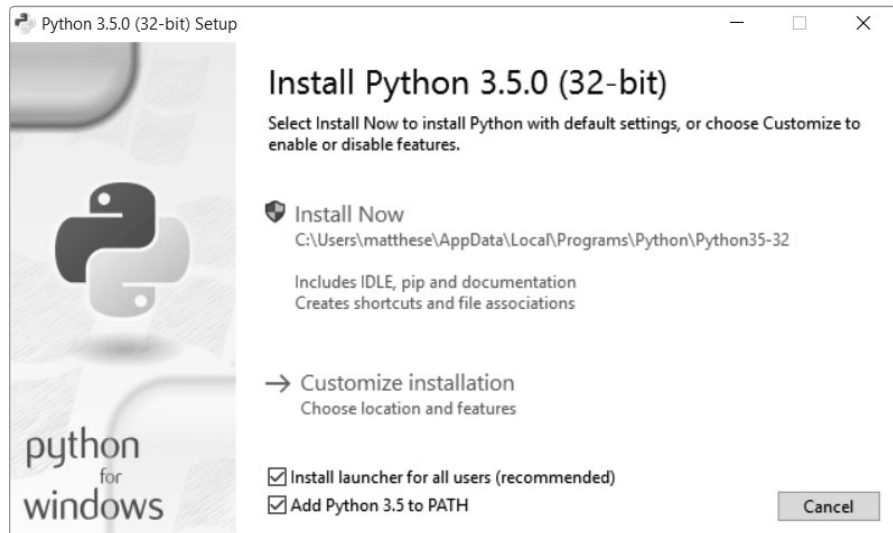


Figure 1-2: Make sure you check the box labeled Add Python to PATH.

Starting a Python Terminal Session

Setting up your text editor will be straightforward if you first set up your system to run Python in a terminal session. Open a command window and enter **python** in lowercase. If you get a Python prompt (**>>>**), Windows has found the version of Python you just installed:

```
C:\> python
Python 3.5.0 (v3.5.0:374f501f4567, Sep 13 2015, 22:15:05) [MSC v.1900 32 bit
(Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

If this worked, you can move on to the next section, “Running Python in a Terminal Session.”

However, you may see output that looks more like this:

```
C:\> python
'python' is not recognized as an internal or external command, operable
program or batch file.
```

In this case you need to tell Windows how to find the Python version you just installed. Your system’s `python` command is usually saved in your *C* drive, so open Windows Explorer and open your *C* drive. Look for a folder starting with the name *Python*, open that folder, and find the *python* file (in lowercase). For example, I have a *Python35* folder with a file named *python* inside it, so the path to the `python` command on my system is *C:\Python35\python*. Otherwise, enter **python** into the search box in Windows Explorer to show you exactly where the `python` command is stored on your system.

When you think you know the path, test it by entering that path into a terminal window. Open a command window and enter the full path you just found:

```
C:\> C:\Python35\python
Python 3.5.0 (v3.5.0:374f501f4567, Sep 13 2015, 22:15:05) [MSC v.1900 32 bit
(Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

If this worked, you know how to access Python on your system.

Running Python in a Terminal Session

Enter the following line in your Python session, and make sure you see the output *Hello Python world!*

```
>>> print("Hello Python world!")
Hello Python world!
>>>
```

Any time you want to run a snippet of Python code, open a command window and start a Python terminal session. To close the terminal session, press CTRL-Z and then press ENTER, or enter the command `exit()`.

Installing a Text Editor

Geany is a simple text editor: it’s easy to install, will let you run almost all of your programs directly from the editor instead of through a terminal, uses syntax highlighting to color your code, and runs your code in a terminal window so you’ll get used to using terminals. Appendix B provides information on other text editors, but I recommend using Geany unless you have a good reason to use a different editor.

You can download a Windows installer for Geany from <http://geany.org/>. Click **Releases** under the Download menu, and look for the *geany-1.25_setup.exe* installer or something similar. Run the installer and accept all the defaults.

To start your first program, open Geany: press the Windows key and search for Geany on your system. You should make a shortcut by dragging the icon to your taskbar or desktop. Make a folder called *python_work* somewhere on your system for your projects. (It's best to use lowercase letters and underscores for spaces in file and folder names, because these are Python naming conventions.) Go back to Geany and save an empty Python file (**File ▶ Save As**) called *hello_world.py* in your *python_work* folder. The extension *.py* tells Geany that your file will contain a Python program. It also tells Geany how to run your program and to highlight the text in a helpful way.

After you've saved your file, type the following line:

```
print("Hello Python world!")
```

If the command `python` worked on your system, you won't have to configure Geany; skip the next section and move on to "Running the Hello World Program" on page 14. If you needed to enter a path like *C:\Python35\python* to start a Python interpreter, follow the directions in the next section to configure Geany for your system.

Configuring Geany

To configure Geany, go to **Build ▶ Set Build Commands**. You should see the words *Compile* and *Execute* with a command next to each. The Compile and Execute commands start with `python` in lowercase, but Geany doesn't know where your system stored the `python` command. You need to add the path you used in the terminal session.

In the Compile and Execute commands, add the drive your `python` command is on and the folder where the `python` command is stored. Your Compile command should look something like this:

```
C:\Python35\python -m py_compile "%f"
```

Your path might be a little different, but make sure the spaces and capitalization match what is shown here.

Your Execute command should look something like this:

```
C:\Python35\python "%f"
```

Again, make sure the spacing and capitalization in your Execute command matches what is shown here. Figure 1-3 shows how these commands should look in Geany's configuration menu.

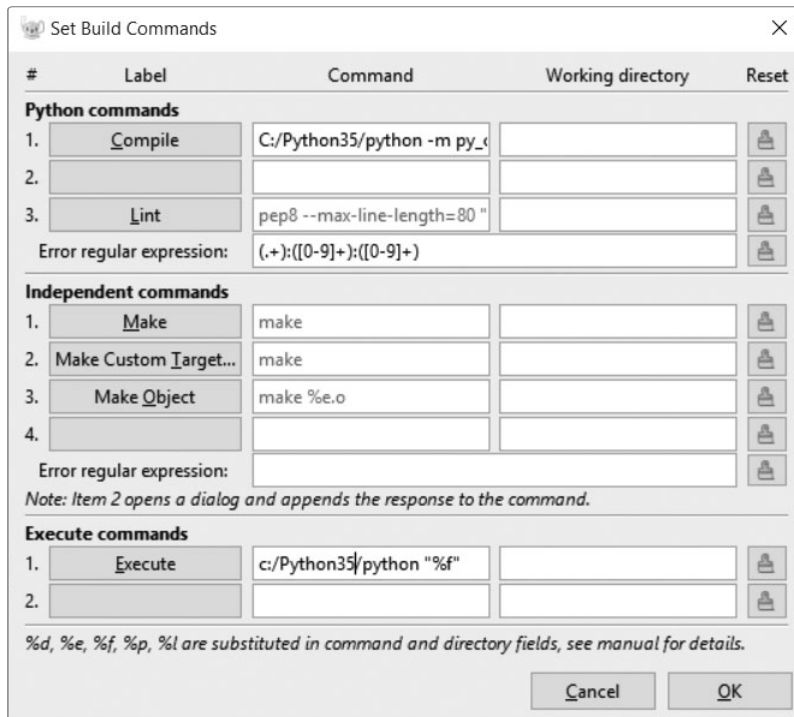


Figure 1-3: Here, Geany is configured to use Python 3 on Windows.

After you've set these commands correctly, click **OK**.

Running the Hello World Program

You should now be able to run your program successfully. Run *hello_world.py* by selecting **Build ► Execute** in the menu, by clicking the Execute icon (which shows a set of gears), or by pressing F5. A terminal window should pop up with the following output:

```
Hello Python world!

-----
(program exited with code: 0)
Press return to continue
```

If you don't see this, check every character on the line you entered. Did you accidentally capitalize `print`? Did you forget one or both of the quotation marks or parentheses? Programming languages expect very specific syntax, and if you don't provide that, you'll get errors. If you can't get the program to run, see the next section for help.

Troubleshooting Installation Issues

Hopefully, setting up your programming environment was successful, but if you've been unable to run `hello_world.py`, here are a few remedies you can try:

- When a program contains a significant error, Python displays a *traceback*. Python looks through the file and tries to report the problem. The traceback might give you a clue as to what issue is preventing the program from running.
- Step away from your computer, take a short break, and then try again. Remember that syntax is very important in programming, so even a missing colon, a mismatched quotation mark, or mismatched parentheses can prevent a program from running properly. Reread the relevant parts of this chapter, look over what you've done, and see if you can find the mistake.
- Start over again. You probably don't need to uninstall anything, but it might make sense to delete your `hello_world.py` file and create it again from scratch.
- Ask someone else to follow the steps in this chapter, on your computer or a different one, and watch what they do carefully. You might have missed one small step that someone else happens to catch.
- Find someone who knows Python and ask them to help you get set up. If you ask around, you might find that you know someone who uses Python.
- The setup instructions in this chapter are also available online, through <https://www.nostarch.com/pythoncrashcourse/>. The online version of these instructions may work better for you.
- Ask for help online. Appendix C provides a number of resources and areas online, like forums and live chat sites, where you can ask for solutions from people who've already worked through the issue you're currently facing.

Don't worry about bothering experienced programmers. Every programmer has been stuck at some point, and most programmers are happy to help you set up your system correctly. As long as you can state clearly what you're trying to do, what you've already tried, and the results you're getting, there's a good chance someone will be able to help you. As mentioned in the Introduction, the Python community is very beginner friendly.

Python should run well on any modern computer, so find a way to ask for help if you're having trouble so far. Early issues can be frustrating, but they're well worth sorting out. Once you get `hello_world.py` running, you can start to learn Python, and your programming work will become more interesting and satisfying.

Running Python Programs from a Terminal

Most of the programs you write in your text editor you'll run directly from the editor, but sometimes it's useful to run programs from a terminal instead. For example, you might want to run an existing program without opening it for editing.

You can do this on any system with Python installed if you know how to access the directory where you've stored your program file. To try this, make sure you've saved the *hello_world.py* file in the *python_work* folder on your desktop.

On Linux and OS X

Running a Python program from a terminal session is the same on Linux and OS X. The terminal command *cd*, for *change directory*, is used to navigate through your file system in a terminal session. The command *ls*, for *list*, shows you all the nonhidden files that exist in the current directory.

Open a new terminal window and issue the following commands to run *hello_world.py*:

```
❶ ~$ cd Desktop/python_work/
❷ ~/Desktop/python_work$ ls
hello_world.py
❸ ~/Desktop/python_work$ python hello_world.py
Hello Python world!
```

At ❶ we use the *cd* command to navigate to the *python_work* folder, which is in the *Desktop* folder. Next, we use the *ls* command to make sure *hello_world.py* is in this folder ❷. Then, we run the file using the command *python hello_world.py* ❸.

It's that simple. You just use the *python* (or *python3*) command to run Python programs.

On Windows

The terminal command *cd*, for *change directory*, is used to navigate through your file system in a command window. The command *dir*, for *directory*, shows you all the files that exist in the current directory.

Open a new terminal window and issue the following commands to run *hello_world.py*:

```
❶ C:\> cd Desktop\python_work
❷ C:\Desktop\python_work> dir
hello_world.py
❸ C:\Desktop\python_work> python hello_world.py
Hello Python world!
```

At ❶ we use the *cd* command to navigate to the *python_work* folder, which is in the *Desktop* folder. Next, we use the *dir* command to make sure *hello_world.py* is in this folder ❷. Then, we run the file using the command *python hello_world.py* ❸.

If you haven't configured your system to use the simple command `python`, you may need to use the longer version of this command:

```
C:\> cd Desktop\python_work
C:\Desktop\python_work> dir
hello_world.py
C:\Desktop\python_work> C:\Python35\python hello_world.py
Hello Python world!
```

Most of your programs will run fine directly from your editor, but as your work becomes more complex, you might write programs that you'll need to run from a terminal.

TRY IT YOURSELF

The exercises in this chapter are exploratory in nature. Starting in Chapter 2, the challenges you'll solve will be based on what you've learned.

1-1. python.org: Explore the Python home page (<http://python.org/>) to find topics that interest you. As you become familiar with Python, different parts of the site will be more useful to you.

1-2. Hello World Typos: Open the `hello_world.py` file you just created. Make a typo somewhere in the line and run the program again. Can you make a typo that generates an error? Can you make sense of the error message? Can you make a typo that doesn't generate an error? Why do you think it didn't make an error?

1-3. Infinite Skills: If you had infinite programming skills, what would you build? You're about to learn how to program. If you have an end goal in mind, you'll have an immediate use for your new skills; now is a great time to draft descriptions of what you'd like to create. It's a good habit to keep an "ideas" notebook that you can refer to whenever you want to start a new project. Take a few minutes now to describe three programs you'd like to create.

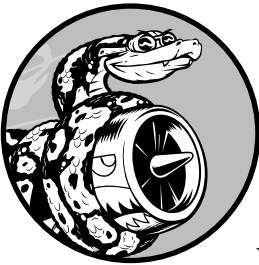
Summary

In this chapter you learned a bit about Python in general, and you installed Python to your system if it wasn't already there. You also installed a text editor to make it easier to write Python code. You learned to run snippets of Python code in a terminal session, and you ran your first actual program, `hello_world.py`. You probably learned a bit about troubleshooting as well.

In the next chapter you'll learn about the different kinds of data you can work with in your Python programs, and you'll learn to use variables as well.

2

VARIABLES AND SIMPLE DATA TYPES



In this chapter you'll learn about the different kinds of data you can work with in your Python programs. You'll also learn how to store your data in variables and how to use those variables in your programs.

What Really Happens When You Run `hello_world.py`

Let's take a closer look at what Python does when you run `hello_world.py`. As it turns out, Python does a fair amount of work, even when it runs a simple program:

<code>hello_world.py</code>	<code>print("Hello Python world!")</code>
-----------------------------	---

When you run this code, you should see this output:

```
Hello Python world!
```

When you run the file *hello_world.py*, the ending *.py* indicates that the file is a Python program. Your editor then runs the file through the *Python interpreter*, which reads through the program and determines what each word in the program means. For example, when the interpreter sees the word `print`, it prints to the screen whatever is inside the parentheses.

As you write your programs, your editor highlights different parts of your program in different ways. For example, it recognizes that `print` is the name of a function and displays that word in blue. It recognizes that “Hello Python world!” is not Python code and displays that phrase in orange. This feature is called *syntax highlighting* and is quite useful as you start to write your own programs.

Variables

Let’s try using a variable in *hello_world.py*. Add a new line at the beginning of the file, and modify the second line:

```
message = "Hello Python world!"  
print(message)
```

Run this program to see what happens. You should see the same output you saw previously:

```
Hello Python world!
```

We’ve added a *variable* named `message`. Every variable holds a *value*, which is the information associated with that variable. In this case the value is the text “Hello Python world!”

Adding a variable makes a little more work for the Python interpreter. When it processes the first line, it associates the text “Hello Python world!” with the variable `message`. When it reaches the second line, it prints the value associated with `message` to the screen.

Let’s expand on this program by modifying *hello_world.py* to print a second message. Add a blank line to *hello_world.py*, and then add two new lines of code:

```
message = "Hello Python world!"  
print(message)  
  
message = "Hello Python Crash Course world!"  
print(message)
```

Now when you run `hello_world.py`, you should see two lines of output:

```
Hello Python world!  
Hello Python Crash Course world!
```

You can change the value of a variable in your program at any time, and Python will always keep track of its current value.

Naming and Using Variables

When you're using variables in Python, you need to adhere to a few rules and guidelines. Breaking some of these rules will cause errors; other guidelines just help you write code that's easier to read and understand. Be sure to keep the following variable rules in mind:

- Variable names can contain only letters, numbers, and underscores. They can start with a letter or an underscore, but not with a number. For instance, you can call a variable `message_1` but not `1_message`.
- Spaces are not allowed in variable names, but underscores can be used to separate words in variable names. For example, `greeting_message` works, but `greeting message` will cause errors.
- Avoid using Python keywords and function names as variable names; that is, do not use words that Python has reserved for a particular programmatic purpose, such as the word `print`. (See “Python Keywords and Built-in Functions” on page 489.)
- Variable names should be short but descriptive. For example, `name` is better than `n`, `student_name` is better than `s_n`, and `name_length` is better than `length_of_persons_name`.
- Be careful when using the lowercase letter `l` and the uppercase letter `O` because they could be confused with the numbers `1` and `0`.

It can take some practice to learn how to create good variable names, especially as your programs become more interesting and complicated. As you write more programs and start to read through other people's code, you'll get better at coming up with meaningful names.

NOTE

The Python variables you're using at this point should be lowercase. You won't get errors if you use uppercase letters, but it's a good idea to avoid using them for now.

Avoiding Name Errors When Using Variables

Every programmer makes mistakes, and most make mistakes every day. Although good programmers might create errors, they also know how to respond to those errors efficiently. Let's look at an error you're likely to make early on and learn how to fix it.

We'll write some code that generates an error on purpose. Enter the following code, including the misspelled word *message* shown in bold:

```
message = "Hello Python Crash Course reader!"  
print(message)
```

When an error occurs in your program, the Python interpreter does its best to help you figure out where the problem is. The interpreter provides a traceback when a program cannot run successfully. A *traceback* is a record of where the interpreter ran into trouble when trying to execute your code. Here's an example of the traceback that Python provides after you've accidentally misspelled a variable's name:

Traceback (most recent call last):

- ❶ File "hello_world.py", line 2, in <module>
 - ❷ print(message)
 - ❸ NameError: name 'message' is not defined
-

The output at ❶ reports that an error occurs in line 2 of the file *hello_world.py*. The interpreter shows this line to help us spot the error quickly ❷ and tells us what kind of error it found ❸. In this case it found a *name error* and reports that the variable being printed, *message*, has not been defined. Python can't identify the variable name provided. A name error usually means we either forgot to set a variable's value before using it, or we made a spelling mistake when entering the variable's name.

Of course, in this example we omitted the letter *s* in the variable name *message* in the second line. The Python interpreter doesn't spellcheck your code, but it does ensure that variable names are spelled consistently. For example, watch what happens when we spell *message* incorrectly in another place in the code as well:

```
message = "Hello Python Crash Course reader!"  
print(message)
```

In this case, the program runs successfully!

```
Hello Python Crash Course reader!
```

Computers are strict, but they disregard good and bad spelling. As a result, you don't need to consider English spelling and grammar rules when you're trying to create variable names and writing code.

Many programming errors are simple, single-character typos in one line of a program. If you're spending a long time searching for one of these errors, know that you're in good company. Many experienced and talented programmers spend hours hunting down these kinds of tiny errors. Try to laugh about it and move on, knowing it will happen frequently throughout your programming life.

NOTE

The best way to understand new programming concepts is to try using them in your programs. If you get stuck while working on an exercise in this book, try doing something else for a while. If you're still stuck, review the relevant part of that chapter. If you still need help, see the suggestions in Appendix C.

TRY IT YOURSELF

Write a separate program to accomplish each of these exercises. Save each program with a filename that follows standard Python conventions, using lowercase letters and underscores, such as `simple_message.py` and `simple_messages.py`.

2-1. Simple Message: Store a message in a variable, and then print that message.

2-2. Simple Messages: Store a message in a variable, and print that message. Then change the value of your variable to a new message, and print the new message.

Strings

Because most programs define and gather some sort of data, and then do something useful with it, it helps to classify different types of data. The first data type we'll look at is the string. Strings are quite simple at first glance, but you can use them in many different ways.

A *string* is simply a series of characters. Anything inside quotes is considered a string in Python, and you can use single or double quotes around your strings like this:

```
"This is a string."  
'This is also a string.'
```

This flexibility allows you to use quotes and apostrophes within your strings:

```
'I told my friend, "Python is my favorite language!'"  
"The language 'Python' is named after Monty Python, not the snake."  
"One of Python's strengths is its diverse and supportive community."
```

Let's explore some of the ways you can use strings.

Changing Case in a String with Methods

One of the simplest tasks you can do with strings is change the case of the words in a string. Look at the following code, and try to determine what's happening:

```
name.py    name = "ada lovelace"
           print(name.title())
```

Save this file as *name.py*, and then run it. You should see this output:

```
Ada Lovelace
```

In this example, the lowercase string "ada lovelace" is stored in the variable `name`. The method `title()` appears after the variable in the `print()` statement. A *method* is an action that Python can perform on a piece of data. The dot (.) after `name` in `name.title()` tells Python to make the `title()` method act on the variable `name`. Every method is followed by a set of parentheses, because methods often need additional information to do their work. That information is provided inside the parentheses. The `title()` function doesn't need any additional information, so its parentheses are empty.

`title()` displays each word in titlecase, where each word begins with a capital letter. This is useful because you'll often want to think of a name as a piece of information. For example, you might want your program to recognize the input values `Ada`, `ADA`, and `ada` as the same name, and display all of them as `Ada`.

Several other useful methods are available for dealing with case as well. For example, you can change a string to all uppercase or all lowercase letters like this:

```
name = "Ada Lovelace"
print(name.upper())
print(name.lower())
```

This will display the following:

```
ADA LOVELACE
ada lovelace
```

The `lower()` method is particularly useful for storing data. Many times you won't want to trust the capitalization that your users provide, so you'll convert strings to lowercase before storing them. Then when you want to display the information, you'll use the case that makes the most sense for each string.

Combining or Concatenating Strings

It's often useful to combine strings. For example, you might want to store a first name and a last name in separate variables, and then combine them when you want to display someone's full name:

```
first_name = "ada"
last_name = "lovelace"
❶ full_name = first_name + " " + last_name

print(full_name)
```

Python uses the plus symbol (+) to combine strings. In this example, we use + to create a full name by combining a `first_name`, a space, and a `last_name` ❶, giving this result:

```
ada lovelace
```

This method of combining strings is called *concatenation*. You can use concatenation to compose complete messages using the information you've stored in a variable. Let's look at an example:

```
first_name = "ada"
last_name = "lovelace"
full_name = first_name + " " + last_name

❶ print("Hello, " + full_name.title() + "!")
```

Here, the full name is used at ❶ in a sentence that greets the user, and the `title()` method is used to format the name appropriately. This code returns a simple but nicely formatted greeting:

```
Hello, Ada Lovelace!
```

You can use concatenation to compose a message and then store the entire message in a variable:

```
first_name = "ada"
last_name = "lovelace"
full_name = first_name + " " + last_name

❶ message = "Hello, " + full_name.title() + "!"
❷ print(message)
```

This code displays the message “Hello, Ada Lovelace!” as well, but storing the message in a variable at ❶ makes the final print statement at ❷ much simpler.

Adding Whitespace to Strings with Tabs or Newlines

In programming, *whitespace* refers to any nonprinting character, such as spaces, tabs, and end-of-line symbols. You can use whitespace to organize your output so it's easier for users to read.

To add a tab to your text, use the character combination `\t` as shown at ❶:

```
>>> print("Python")
Python
❶ >>> print("\tPython")
Python
```

To add a newline in a string, use the character combination `\n`:

```
>>> print("Languages:\nPython\nC\nJavaScript")
Languages:
Python
C
JavaScript
```

You can also combine tabs and newlines in a single string. The string `"\n\t"` tells Python to move to a new line, and start the next line with a tab. The following example shows how you can use a one-line string to generate four lines of output:

```
>>> print("Languages:\n\tPython\n\tC\n\tJavaScript")
Languages:
Python
C
JavaScript
```

Newlines and tabs will be very useful in the next two chapters when you start to produce many lines of output from just a few lines of code.

Stripping Whitespace

Extra whitespace can be confusing in your programs. To programmers `'python'` and `'python '` look pretty much the same. But to a program, they are two different strings. Python detects the extra space in `'python '` and considers it significant unless you tell it otherwise.

It's important to think about whitespace, because often you'll want to compare two strings to determine whether they are the same. For example, one important instance might involve checking people's usernames when they log in to a website. Extra whitespace can be confusing in much simpler situations as well. Fortunately, Python makes it easy to eliminate extraneous whitespace from data that people enter.

Python can look for extra whitespace on the right and left sides of a string. To ensure that no whitespace exists at the right end of a string, use the `rstrip()` method.

```
❶ >>> favorite_language = 'python '  
❷ >>> favorite_language  
'python '  
❸ >>> favorite_language.rstrip()  
'python'  
❹ >>> favorite_language  
'python '
```

The value stored in `favorite_language` at ❶ contains extra whitespace at the end of the string. When you ask Python for this value in a terminal session, you can see the space at the end of the value ❷. When the `rstrip()` method acts on the variable `favorite_language` at ❸, this extra space is removed. However, it is only removed temporarily. If you ask for the value of `favorite_language` again, you can see that the string looks the same as when it was entered, including the extra whitespace ❹.

To remove the whitespace from the string permanently, you have to store the stripped value back into the variable:

```
>>> favorite_language = 'python '  
❶ >>> favorite_language = favorite_language.rstrip()  
>>> favorite_language  
'python'
```

To remove the whitespace from the string, you strip the whitespace from the right side of the string and then store that value back in the original variable, as shown at ❶. Changing a variable's value and then storing the new value back in the original variable is done often in programming. This is how a variable's value can change as a program is executed or in response to user input.

You can also strip whitespace from the left side of a string using the `lstrip()` method or strip whitespace from both sides at once using `strip()`:

```
❶ >>> favorite_language = ' python '  
❷ >>> favorite_language.rstrip()  
' python '  
❸ >>> favorite_language.lstrip()  
'python '  
❹ >>> favorite_language.strip()  
'python'
```

In this example, we start with a value that has whitespace at the beginning and the end ❶. We then remove the extra space from the right side at ❷, from the left side at ❸, and from both sides at ❹. Experimenting with these stripping functions can help you become familiar with manipulating strings. In the real world, these stripping functions are used most often to clean up user input before it's stored in a program.

Avoiding Syntax Errors with Strings

One kind of error that you might see with some regularity is a syntax error. A *syntax error* occurs when Python doesn't recognize a section of your program as valid Python code. For example, if you use an apostrophe within single quotes, you'll produce an error. This happens because Python interprets everything between the first single quote and the apostrophe as a string. It then tries to interpret the rest of the text as Python code, which causes errors.

Here's how to use single and double quotes correctly. Save this program as *apostrophe.py* and then run it:

apostrophe.py

```
message = "One of Python's strengths is its diverse community."  
print(message)
```

The apostrophe appears inside a set of double quotes, so the Python interpreter has no trouble reading the string correctly:

```
One of Python's strengths is its diverse community.
```

However, if you use single quotes, Python can't identify where the string should end:

```
message = 'One of Python's strengths is its diverse community.'  
print(message)
```

You'll see the following output:

```
File "apostrophe.py", line 1  
    message = 'One of Python's strengths is its diverse community.'  
                ^❶  
SyntaxError: invalid syntax
```

In the output you can see that the error occurs at ❶ right after the second single quote. This *syntax error* indicates that the interpreter doesn't recognize something in the code as valid Python code. Errors can come from a variety of sources, and I'll point out some common ones as they arise. You might see syntax errors often as you learn to write proper Python code. Syntax errors are also the least specific kind of error, so they can be difficult and frustrating to identify and correct. If you get stuck on a particularly stubborn error, see the suggestions in Appendix C.

NOTE

Your editor's syntax highlighting feature should help you spot some syntax errors quickly as you write your programs. If you see Python code highlighted as if it's English or English highlighted as if it's Python code, you probably have a mismatched quotation mark somewhere in your file.

Printing in Python 2

The print statement has a slightly different syntax in Python 2:

```
>>> python2.7
>>> print "Hello Python 2.7 world!"
Hello Python 2.7 world!
```

Parentheses are not needed around the phrase you want to print in Python 2. Technically, print is a function in Python 3, which is why it needs parentheses. Some Python 2 print statements do include parentheses, but the behavior can be a little different than what you'll see in Python 3. Basically, when you're looking at code written in Python 2, expect to see some print statements with parentheses and some without.

TRY IT YOURSELF

Save each of the following exercises as a separate file with a name like *name_cases.py*. If you get stuck, take a break or see the suggestions in Appendix C.

2-3. Personal Message: Store a person's name in a variable, and print a message to that person. Your message should be simple, such as, "Hello Eric, would you like to learn some Python today?"

2-4. Name Cases: Store a person's name in a variable, and then print that person's name in lowercase, uppercase, and titlecase.

2-5. Famous Quote: Find a quote from a famous person you admire. Print the quote and the name of its author. Your output should look something like the following, including the quotation marks:

Albert Einstein once said, "A person who never made a mistake never tried anything new."

2-6. Famous Quote 2: Repeat Exercise 2-5, but this time store the famous person's name in a variable called *famous_person*. Then compose your message and store it in a new variable called *message*. Print your message.

2-7. Stripping Names: Store a person's name, and include some whitespace characters at the beginning and end of the name. Make sure you use each character combination, "\t" and "\n", at least once.

Print the name once, so the whitespace around the name is displayed. Then print the name using each of the three stripping functions, *lstrip()*, *rstrip()*, and *strip()*.

Numbers

Numbers are used quite often in programming to keep score in games, represent data in visualizations, store information in web applications, and so on. Python treats numbers in several different ways, depending on how they are being used. Let's first look at how Python manages integers, because they are the simplest to work with.

Integers

You can add (+), subtract (-), multiply (*), and divide (/) integers in Python.

```
>>> 2 + 3
5
>>> 3 - 2
1
>>> 2 * 3
6
>>> 3 / 2
1.5
```

In a terminal session, Python simply returns the result of the operation. Python uses two multiplication symbols to represent exponents:

```
>>> 3 ** 2
9
>>> 3 ** 3
27
>>> 10 ** 6
1000000
```

Python supports the order of operations too, so you can use multiple operations in one expression. You can also use parentheses to modify the order of operations so Python can evaluate your expression in the order you specify. For example:

```
>>> 2 + 3*4
14
>>> (2 + 3) * 4
20
```

The spacing in these examples has no effect on how Python evaluates the expressions; it simply helps you more quickly spot the operations that have priority when you're reading through the code.

Floats

Python calls any number with a decimal point a *float*. This term is used in most programming languages, and it refers to the fact that a decimal point can appear at any position in a number. Every programming language must

be carefully designed to properly manage decimal numbers so numbers behave appropriately no matter where the decimal point appears.

For the most part, you can use decimals without worrying about how they behave. Simply enter the numbers you want to use, and Python will most likely do what you expect:

```
>>> 0.1 + 0.1
0.2
>>> 0.2 + 0.2
0.4
>>> 2 * 0.1
0.2
>>> 2 * 0.2
0.4
```

But be aware that you can sometimes get an arbitrary number of decimal places in your answer:

```
>>> 0.2 + 0.1
0.30000000000000004
>>> 3 * 0.1
0.30000000000000004
```

This happens in all languages and is of little concern. Python tries to find a way to represent the result as precisely as possible, which is sometimes difficult given how computers have to represent numbers internally. Just ignore the extra decimal places for now; you'll learn ways to deal with the extra places when you need to in the projects in Part II.

Avoiding Type Errors with the `str()` Function

Often, you'll want to use a variable's value within a message. For example, say you want to wish someone a happy birthday. You might write code like this:

```
birthday.py age = 23
message = "Happy " + age + "rd Birthday!"

print(message)
```

You might expect this code to print the simple birthday greeting, Happy 23rd birthday! But if you run this code, you'll see that it generates an error:

```
Traceback (most recent call last):
  File "birthday.py", line 2, in <module>
    message = "Happy " + age + "rd Birthday!"
❶ TypeError: Can't convert 'int' object to str implicitly
```

This is a *type error*. It means Python can't recognize the kind of information you're using. In this example Python sees at ❶ that you're using a variable that has an integer value (`int`), but it's not sure how to interpret that

value. Python knows that the variable could represent either the numerical value 23 or the characters 2 and 3. When you use integers within strings like this, you need to specify explicitly that you want Python to use the integer as a string of characters. You can do this by wrapping the variable in the `str()` function, which tells Python to represent non-string values as strings:

```
age = 23
message = "Happy " + str(age) + "rd Birthday!"

print(message)
```

Python now knows that you want to convert the numerical value 23 to a string and display the characters 2 and 3 as part of the birthday message. Now you get the message you were expecting, without any errors:

```
Happy 23rd Birthday!
```

Working with numbers in Python is straightforward most of the time. If you're getting unexpected results, check whether Python is interpreting your numbers the way you want it to, either as a numerical value or as a string value.

Integers in Python 2

Python 2 returns a slightly different result when you divide two integers:

```
>>> python2.7
>>> 3 / 2
1
```

Instead of 1.5, Python returns 1. Division of integers in Python 2 results in an integer with the remainder truncated. Note that the result is not a rounded integer; the remainder is simply omitted.

To avoid this behavior in Python 2, make sure that at least one of the numbers is a float. By doing so, the result will be a float as well:

```
>>> 3 / 2
1
>>> 3.0 / 2
1.5
>>> 3 / 2.0
1.5
>>> 3.0 / 2.0
1.5
```

This division behavior is a common source of confusion when people who are used to Python 3 start using Python 2, or vice versa. If you use or create code that mixes integers and floats, watch out for irregular behavior.

TRY IT YOURSELF

2-8. Number Eight: Write addition, subtraction, multiplication, and division operations that each result in the number 8. Be sure to enclose your operations in print statements to see the results. You should create four lines that look like this:

```
print(5 + 3)
```

Your output should simply be four lines with the number 8 appearing once on each line.

2-9. Favorite Number: Store your favorite number in a variable. Then, using that variable, create a message that reveals your favorite number. Print that message.

Comments

Comments are an extremely useful feature in most programming languages. Everything you've written in your programs so far is Python code. As your programs become longer and more complicated, you should add notes within your programs that describe your overall approach to the problem you're solving. A *comment* allows you to write notes in English within your programs.

How Do You Write Comments?

In Python, the hash mark (#) indicates a comment. Anything following a hash mark in your code is ignored by the Python interpreter. For example:

comment.py

```
# Say hello to everyone.  
print("Hello Python people!")
```

Python ignores the first line and executes the second line.

```
Hello Python people!
```

What Kind of Comments Should You Write?

The main reason to write comments is to explain what your code is supposed to do and how you are making it work. When you're in the middle of working on a project, you understand how all of the pieces fit together. But when you return to a project after some time away, you'll likely have forgotten some of the details. You can always study your code for a while and figure out how segments were supposed to work, but writing good comments can save you time by summarizing your overall approach in clear English.

If you want to become a professional programmer or collaborate with other programmers, you should write meaningful comments. Today, most software is written collaboratively, whether by a group of employees at one company or a group of people working together on an open source project. Skilled programmers expect to see comments in code, so it's best to start adding descriptive comments to your programs now. Writing clear, concise comments in your code is one of the most beneficial habits you can form as a new programmer.

When you're determining whether to write a comment, ask yourself if you had to consider several approaches before coming up with a reasonable way to make something work; if so, write a comment about your solution. It's much easier to delete extra comments later on than it is to go back and write comments for a sparsely commented program. From now on, I'll use comments in examples throughout this book to help explain sections of code.

TRY IT YOURSELF

2-10. Adding Comments: Choose two of the programs you've written, and add at least one comment to each. If you don't have anything specific to write because your programs are too simple at this point, just add your name and the current date at the top of each program file. Then write one sentence describing what the program does.

The Zen of Python

For a long time, the programming language Perl was the mainstay of the Internet. Most interactive websites in the early days were powered by Perl scripts. The Perl community's motto at the time was, "There's more than one way to do it." People liked this mind-set for a while, because the flexibility written into the language made it possible to solve most problems in a variety of ways. This approach was acceptable while working on your own projects, but eventually people realized that the emphasis on flexibility made it difficult to maintain large projects over long periods of time. It was difficult, tedious, and time-consuming to review code and try to figure out what someone else was thinking when they were solving a complex problem.

Experienced Python programmers will encourage you to avoid complexity and aim for simplicity whenever possible. The Python community's philosophy is contained in "The Zen of Python" by Tim Peters. You can access this brief set of principles for writing good Python code by entering `import this` into your interpreter. I won't reproduce the entire "Zen of

Python” here, but I’ll share a few lines to help you understand why they should be important to you as a beginning Python programmer.

```
>>> import this
```

The Zen of Python, by Tim Peters

Beautiful is better than ugly.

Python programmers embrace the notion that code can be beautiful and elegant. In programming, people solve problems. Programmers have always respected well-designed, efficient, and even beautiful solutions to problems. As you learn more about Python and use it to write more code, someone might look over your shoulder one day and say, “Wow, that’s some beautiful code!”

Simple is better than complex.

If you have a choice between a simple and a complex solution, and both work, use the simple solution. Your code will be easier to maintain, and it will be easier for you and others to build on that code later on.

Complex is better than complicated.

Real life is messy, and sometimes a simple solution to a problem is unattainable. In that case, use the simplest solution that works.

Readability counts.

Even when your code is complex, aim to make it readable. When you’re working on a project that involves complex coding, focus on writing informative comments for that code.

There should be one-- and preferably only one --obvious way to do it.

If two Python programmers are asked to solve the same problem, they should come up with fairly compatible solutions. This is not to say there’s no room for creativity in programming. On the contrary! But much of programming consists of using small, common approaches to simple situations within a larger, more creative project. The nuts and bolts of your programs should make sense to other Python programmers.

Now is better than never.

You could spend the rest of your life learning all the intricacies of Python and of programming in general, but then you’d never complete any projects. Don’t try to write perfect code; write code that works, and then decide whether to improve your code for that project or move on to something new.

As you continue to the next chapter and start digging into more involved topics, try to keep this philosophy of simplicity and clarity in mind. Experienced programmers will respect your code more and will be happy to give you feedback and collaborate with you on interesting projects.

TRY IT YOURSELF

2-11. Zen of Python: Enter `import this` into a Python terminal session and skim through the additional principles.

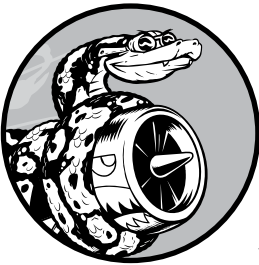
Summary

In this chapter you learned to work with variables. You learned to use descriptive variable names and how to resolve name errors and syntax errors when they arise. You learned what strings are and how to display strings using lowercase, uppercase, and titlecase. You started using whitespace to organize output neatly, and you learned to strip unneeded whitespace from different parts of a string. You started working with integers and floats, and you read about some unexpected behavior to watch out for when working with numerical data. You also learned to write explanatory comments to make your code easier for you and others to read. Finally, you read about the philosophy of keeping your code as simple as possible, whenever possible.

In Chapter 3 you'll learn to store collections of information in variables called *lists*. You'll learn to work through a list, manipulating any information in that list.

3

INTRODUCING LISTS



In this chapter and the next you'll learn what lists are and how to start working with the elements in a list. Lists allow you to store sets of information in one place, whether you have just a few items or millions of items. Lists are one of Python's most powerful features readily accessible to new programmers, and they tie together many important concepts in programming.

What Is a List?

A *list* is a collection of items in a particular order. You can make a list that includes the letters of the alphabet, the digits from 0–9, or the names of all the people in your family. You can put anything you want into a list, and

the items in your list don't have to be related in any particular way. Because a list usually contains more than one element, it's a good idea to make the name of your list plural, such as letters, digits, or names.

In Python, square brackets (`[]`) indicate a list, and individual elements in the list are separated by commas. Here's a simple example of a list that contains a few kinds of bicycles:

```
bicycles.py bicycles = ['trek', 'cannondale', 'redline', 'specialized']  
print(bicycles)
```

If you ask Python to print a list, Python returns its representation of the list, including the square brackets:

```
['trek', 'cannondale', 'redline', 'specialized']
```

Because this isn't the output you want your users to see, let's learn how to access the individual items in a list.

Accessing Elements in a List

Lists are ordered collections, so you can access any element in a list by telling Python the position, or *index*, of the item desired. To access an element in a list, write the name of the list followed by the index of the item enclosed in square brackets.

For example, let's pull out the first bicycle in the list `bicycles`:

```
❶ bicycles = ['trek', 'cannondale', 'redline', 'specialized']  
print(bicycles[0])
```

The syntax for this is shown at ❶. When we ask for a single item from a list, Python returns just that element without square brackets or quotation marks:

```
trek
```

This is the result you want your users to see—clean, neatly formatted output.

You can also use the string methods from Chapter 2 on any element in a list. For example, you can format the element `'trek'` more neatly by using the `title()` method:

```
bicycles = ['trek', 'cannondale', 'redline', 'specialized']  
print(bicycles[0].title())
```

This example produces the same output as the preceding example except `'Trek'` is capitalized.

Index Positions Start at 0, Not 1

Python considers the first item in a list to be at position 0, not position 1. This is true of most programming languages, and the reason has to do with how the list operations are implemented at a lower level. If you're receiving unexpected results, determine whether you are making a simple off-by-one error.

The second item in a list has an index of 1. Using this simple counting system, you can get any element you want from a list by subtracting one from its position in the list. For instance, to access the fourth item in a list, you request the item at index 3.

The following asks for the bicycles at index 1 and index 3:

```
bicycles = ['trek', 'cannondale', 'redline', 'specialized']  
print(bicycles[1])  
print(bicycles[3])
```

This code returns the second and fourth bicycles in the list:

```
cannondale  
specialized
```

Python has a special syntax for accessing the last element in a list. By asking for the item at index -1, Python always returns the last item in the list:

```
bicycles = ['trek', 'cannondale', 'redline', 'specialized']  
print(bicycles[-1])
```

This code returns the value 'specialized'. This syntax is quite useful, because you'll often want to access the last items in a list without knowing exactly how long the list is. This convention extends to other negative index values as well. The index -2 returns the second item from the end of the list, the index -3 returns the third item from the end, and so forth.

Using Individual Values from a List

You can use individual values from a list just as you would any other variable. For example, you can use concatenation to create a message based on a value from a list.

Let's try pulling the first bicycle from the list and composing a message using that value.

```
bicycles = ['trek', 'cannondale', 'redline', 'specialized']  
❶ message = "My first bicycle was a " + bicycles[0].title() + ". "  
  
print(message)
```

At ❶, we build a sentence using the value at `bicycles[0]` and store it in the variable `message`. The output is a simple sentence about the first bicycle in the list:

My first bicycle was a Trek.

TRY IT YOURSELF

Try these short programs to get some firsthand experience with Python's lists. You might want to create a new folder for each chapter's exercises to keep them organized.

3-1. Names: Store the names of a few of your friends in a list called `names`. Print each person's name by accessing each element in the list, one at a time.

3-2. Greetings: Start with the list you used in Exercise 3-1, but instead of just printing each person's name, print a message to them. The text of each message should be the same, but each message should be personalized with the person's name.

3-3. Your Own List: Think of your favorite mode of transportation, such as a motorcycle or a car, and make a list that stores several examples. Use your list to print a series of statements about these items, such as "I would like to own a Honda motorcycle."

Changing, Adding, and Removing Elements

Most lists you create will be dynamic, meaning you'll build a list and then add and remove elements from it as your program runs its course. For example, you might create a game in which a player has to shoot aliens out of the sky. You could store the initial set of aliens in a list and then remove an alien from the list each time one is shot down. Each time a new alien appears on the screen, you add it to the list. Your list of aliens will decrease and increase in length throughout the course of the game.

Modifying Elements in a List

The syntax for modifying an element is similar to the syntax for accessing an element in a list. To change an element, use the name of the list followed by the index of the element you want to change, and then provide the new value you want that item to have.

For example, let's say we have a list of motorcycles, and the first item in the list is 'honda'. How would we change the value of this first item?

```
motorcycles.py ❶ motorcycles = ['honda', 'yamaha', 'suzuki']  
                 print(motorcycles)  
  
                 ❷ motorcycles[0] = 'ducati'  
                 print(motorcycles)
```

The code at ❶ defines the original list, with 'honda' as the first element. The code at ❷ changes the value of the first item to 'ducati'. The output shows that the first item has indeed been changed, and the rest of the list stays the same:

```
['honda', 'yamaha', 'suzuki']  
['ducati', 'yamaha', 'suzuki']
```

You can change the value of any item in a list, not just the first item.

Adding Elements to a List

You might want to add a new element to a list for many reasons. For example, you might want to make new aliens appear in a game, add new data to a visualization, or add new registered users to a website you've built. Python provides several ways to add new data to existing lists.

Appending Elements to the End of a List

The simplest way to add a new element to a list is to *append* the item to the list. When you append an item to a list, the new element is added to the end of the list. Using the same list we had in the previous example, we'll add the new element 'ducati' to the end of the list:

```
motorcycles = ['honda', 'yamaha', 'suzuki']  
print(motorcycles)  
  
❶ motorcycles.append('ducati')  
   print(motorcycles)
```

The `append()` method at ❶ adds 'ducati' to the end of the list without affecting any of the other elements in the list:

```
['honda', 'yamaha', 'suzuki']  
['honda', 'yamaha', 'suzuki', 'ducati']
```

The `append()` method makes it easy to build lists dynamically. For example, you can start with an empty list and then add items to the list using a series of `append()` statements. Using an empty list, let's add the elements 'honda', 'yamaha', and 'suzuki' to the list:

```
motorcycles = []

motorcycles.append('honda')
motorcycles.append('yamaha')
motorcycles.append('suzuki')

print(motorcycles)
```

The resulting list looks exactly the same as the lists in the previous examples:

```
['honda', 'yamaha', 'suzuki']
```

Building lists this way is very common, because you often won't know the data your users want to store in a program until after the program is running. To put your users in control, start by defining an empty list that will hold the users' values. Then append each new value provided to the list you just created.

Inserting Elements into a List

You can add a new element at any position in your list by using the `insert()` method. You do this by specifying the index of the new element and the value of the new item.

```
motorcycles = ['honda', 'yamaha', 'suzuki']

❶ motorcycles.insert(0, 'ducati')
print(motorcycles)
```

In this example, the code at ❶ inserts the value 'ducati' at the beginning of the list. The `insert()` method opens a space at position 0 and stores the value 'ducati' at that location. This operation shifts every other value in the list one position to the right:

```
['ducati', 'honda', 'yamaha', 'suzuki']
```

Removing Elements from a List

Often, you'll want to remove an item or a set of items from a list. For example, when a player shoots down an alien from the sky, you'll most likely want to remove it from the list of active aliens. Or when a user

decides to cancel their account on a web application you created, you'll want to remove that user from the list of active users. You can remove an item according to its position in the list or according to its value.

Removing an Item Using the `del` Statement

If you know the position of the item you want to remove from a list, you can use the `del` statement.

```
motorcycles = ['honda', 'yamaha', 'suzuki']  
print(motorcycles)
```

```
❶ del motorcycles[0]  
print(motorcycles)
```

The code at ❶ uses `del` to remove the first item, 'honda', from the list of motorcycles:

```
['honda', 'yamaha', 'suzuki']  
['yamaha', 'suzuki']
```

You can remove an item from any position in a list using the `del` statement if you know its index. For example, here's how to remove the second item, 'yamaha', in the list:

```
motorcycles = ['honda', 'yamaha', 'suzuki']  
print(motorcycles)
```

```
del motorcycles[1]  
print(motorcycles)
```

The second motorcycle is deleted from the list:

```
['honda', 'yamaha', 'suzuki']  
['honda', 'suzuki']
```

In both examples, you can no longer access the value that was removed from the list after the `del` statement is used.

Removing an Item Using the `pop()` Method

Sometimes you'll want to use the value of an item after you remove it from a list. For example, you might want to get the *x* and *y* position of an alien that was just shot down, so you can draw an explosion at that position. In a web application, you might want to remove a user from a list of active members and then add that user to a list of inactive members.

The `pop()` method removes the last item in a list, but it lets you work with that item after removing it. The term *pop* comes from thinking of a list as a stack of items and popping one item off the top of the stack. In this analogy, the top of a stack corresponds to the end of a list.

Let's pop a motorcycle from the list of motorcycles:

```
❶ motorcycles = ['honda', 'yamaha', 'suzuki']
   print(motorcycles)

❷ popped_motorcycle = motorcycles.pop()
❸ print(motorcycles)
❹ print(popped_motorcycle)
```

We start by defining and printing the list `motorcycles` at ❶. At ❷ we pop a value from the list and store that value in the variable `popped_motorcycle`. We print the list at ❸ to show that a value has been removed from the list. Then we print the popped value at ❹ to prove that we still have access to the value that was removed.

The output shows that the value 'suzuki' was removed from the end of the list and is now stored in the variable `popped_motorcycle`:

```
['honda', 'yamaha', 'suzuki']
['honda', 'yamaha']
suzuki
```

How might this `pop()` method be useful? Imagine that the motorcycles in the list are stored in chronological order according to when we owned them. If this is the case, we can use the `pop()` method to print a statement about the last motorcycle we bought:

```
motorcycles = ['honda', 'yamaha', 'suzuki']

last_owned = motorcycles.pop()
print("The last motorcycle I owned was a " + last_owned.title() + ".")
```

The output is a simple sentence about the most recent motorcycle we owned:

```
The last motorcycle I owned was a Suzuki.
```

Popping Items from any Position in a List

You can actually use `pop()` to remove an item in a list at any position by including the index of the item you want to remove in parentheses.

```
motorcycles = ['honda', 'yamaha', 'suzuki']

❶ first_owned = motorcycles.pop(0)
❷ print('The first motorcycle I owned was a ' + first_owned.title() + '.')
```

We start by popping the first motorcycle in the list at ❶, and then we print a message about that motorcycle at ❷. The output is a simple sentence describing the first motorcycle I ever owned:

```
The first motorcycle I owned was a Honda.
```

Remember that each time you use `pop()`, the item you work with is no longer stored in the list.

If you're unsure whether to use the `del` statement or the `pop()` method, here's a simple way to decide: when you want to delete an item from a list and not use that item in any way, use the `del` statement; if you want to use an item as you remove it, use the `pop()` method.

Removing an Item by Value

Sometimes you won't know the position of the value you want to remove from a list. If you only know the value of the item you want to remove, you can use the `remove()` method.

For example, let's say we want to remove the value 'ducati' from the list of motorcycles.

```
motorcycles = ['honda', 'yamaha', 'suzuki', 'ducati']  
print(motorcycles)
```

```
❶ motorcycles.remove('ducati')  
print(motorcycles)
```

The code at ❶ tells Python to figure out where 'ducati' appears in the list and remove that element:

```
['honda', 'yamaha', 'suzuki', 'ducati']  
['honda', 'yamaha', 'suzuki']
```

You can also use the `remove()` method to work with a value that's being removed from a list. Let's remove the value 'ducati' and print a reason for removing it from the list:

```
❶ motorcycles = ['honda', 'yamaha', 'suzuki', 'ducati']  
print(motorcycles)  
  
❷ too_expensive = 'ducati'  
❸ motorcycles.remove(too_expensive)  
print(motorcycles)  
❹ print("\nA " + too_expensive.title() + " is too expensive for me.")
```

After defining the list at ❶, we store the value 'ducati' in a variable called `too_expensive` ❷. We then use this variable to tell Python which value

to remove from the list at ❸. At ❹ the value 'ducati' has been removed from the list but is still stored in the variable `too_expensive`, allowing us to print a statement about why we removed 'ducati' from the list of motorcycles:

```
['honda', 'yamaha', 'suzuki', 'ducati']  
['honda', 'yamaha', 'suzuki']
```

```
A Ducati is too expensive for me.
```

NOTE

The `remove()` method deletes only the first occurrence of the value you specify. If there's a possibility the value appears more than once in the list, you'll need to use a loop to determine if all occurrences of the value have been removed. You'll learn how to do this in Chapter 7.

TRY IT YOURSELF

The following exercises are a bit more complex than those in Chapter 2, but they give you an opportunity to use lists in all of the ways described.

3-4. Guest List: If you could invite anyone, living or deceased, to dinner, who would you invite? Make a list that includes at least three people you'd like to invite to dinner. Then use your list to print a message to each person, inviting them to dinner.

3-5. Changing Guest List: You just heard that one of your guests can't make the dinner, so you need to send out a new set of invitations. You'll have to think of someone else to invite.

- Start with your program from Exercise 3-4. Add a print statement at the end of your program stating the name of the guest who can't make it.
- Modify your list, replacing the name of the guest who can't make it with the name of the new person you are inviting.
- Print a second set of invitation messages, one for each person who is still in your list.

3-6. More Guests: You just found a bigger dinner table, so now more space is available. Think of three more guests to invite to dinner.

- Start with your program from Exercise 3-4 or Exercise 3-5. Add a print statement to the end of your program informing people that you found a bigger dinner table.
- Use `insert()` to add one new guest to the beginning of your list.
- Use `insert()` to add one new guest to the middle of your list.
- Use `append()` to add one new guest to the end of your list.
- Print a new set of invitation messages, one for each person in your list.

3-7. Shrinking Guest List: You just found out that your new dinner table won't arrive in time for the dinner, and you have space for only two guests.

- Start with your program from Exercise 3-6. Add a new line that prints a message saying that you can invite only two people for dinner.
- Use `pop()` to remove guests from your list one at a time until only two names remain in your list. Each time you pop a name from your list, print a message to that person letting them know you're sorry you can't invite them to dinner.
- Print a message to each of the two people still on your list, letting them know they're still invited.
- Use `del` to remove the last two names from your list, so you have an empty list. Print your list to make sure you actually have an empty list at the end of your program.

Organizing a List

Often, your lists will be created in an unpredictable order, because you can't always control the order in which your users provide their data. Although this is unavoidable in most circumstances, you'll frequently want to present your information in a particular order. Sometimes you'll want to preserve the original order of your list, and other times you'll want to change the original order. Python provides a number of different ways to organize your lists, depending on the situation.

Sorting a List Permanently with the `sort()` Method

Python's `sort()` method makes it relatively easy to sort a list. Imagine we have a list of cars and want to change the order of the list to store them alphabetically. To keep the task simple, let's assume that all the values in the list are lowercase.

```
cars.py cars = ['bmw', 'audi', 'toyota', 'subaru']
❶ cars.sort()
print(cars)
```

The `sort()` method, shown at ❶, changes the order of the list permanently. The cars are now in alphabetical order, and we can never revert to the original order:

```
['audi', 'bmw', 'subaru', 'toyota']
```

You can also sort this list in reverse alphabetical order by passing the argument `reverse=True` to the `sort()` method. The following example sorts the list of cars in reverse alphabetical order:

```
cars = ['bmw', 'audi', 'toyota', 'subaru']
cars.sort(reverse=True)
print(cars)
```

Again, the order of the list is permanently changed:

```
['toyota', 'subaru', 'bmw', 'audi']
```

Sorting a List Temporarily with the `sorted()` Function

To maintain the original order of a list but present it in a sorted order, you can use the `sorted()` function. The `sorted()` function lets you display your list in a particular order but doesn't affect the actual order of the list.

Let's try this function on the list of cars.

```
cars = ['bmw', 'audi', 'toyota', 'subaru']
```

- ```
❶ print("Here is the original list:")
 print(cars)

❷ print("\nHere is the sorted list:")
 print(sorted(cars))

❸ print("\nHere is the original list again:")
 print(cars)
```
- 

We first print the list in its original order at ❶ and then in alphabetical order at ❷. After the list is displayed in the new order, we show that the list is still stored in its original order at ❸.

---

```
Here is the original list:
['bmw', 'audi', 'toyota', 'subaru']
```

```
Here is the sorted list:
['audi', 'bmw', 'subaru', 'toyota']
```

- ```
❹ Here is the original list again:
['bmw', 'audi', 'toyota', 'subaru']
```
-

Notice that the list still exists in its original order at ❹ after the `sorted()` function has been used. The `sorted()` function can also accept a `reverse=True` argument if you want to display a list in reverse alphabetical order.

NOTE

Sorting a list alphabetically is a bit more complicated when all the values are not in lowercase. There are several ways to interpret capital letters when you're deciding on a sort order, and specifying the exact order can be more complex than we want to deal with at this time. However, most approaches to sorting will build directly on what you learned in this section.

Printing a List in Reverse Order

To reverse the original order of a list, you can use the `reverse()` method. If we originally stored the list of cars in chronological order according to when we owned them, we could easily rearrange the list into reverse chronological order:

```
cars = ['bmw', 'audi', 'toyota', 'subaru']  
print(cars)
```

```
cars.reverse()  
print(cars)
```

Notice that `reverse()` doesn't sort backward alphabetically; it simply reverses the order of the list:

```
['bmw', 'audi', 'toyota', 'subaru']  
['subaru', 'toyota', 'audi', 'bmw']
```

The `reverse()` method changes the order of a list permanently, but you can revert to the original order anytime by applying `reverse()` to the same list a second time.

Finding the Length of a List

You can quickly find the length of a list by using the `len()` function. The list in this example has four items, so its length is 4:

```
>>> cars = ['bmw', 'audi', 'toyota', 'subaru']  
>>> len(cars)  
4
```

You'll find `len()` useful when you need to identify the number of aliens that still need to be shot down in a game, determine the amount of data you have to manage in a visualization, or figure out the number of registered users on a website, among other tasks.

NOTE

Python counts the items in a list starting with one, so you shouldn't run into any off-by-one errors when determining the length of a list.

TRY IT YOURSELF

3-8. Seeing the World: Think of at least five places in the world you'd like to visit.

- Store the locations in a list. Make sure the list is not in alphabetical order.
- Print your list in its original order. Don't worry about printing the list neatly, just print it as a raw Python list.
- Use `sorted()` to print your list in alphabetical order without modifying the actual list.
- Show that your list is still in its original order by printing it.
- Use `sorted()` to print your list in reverse alphabetical order without changing the order of the original list.
- Show that your list is still in its original order by printing it again.
- Use `reverse()` to change the order of your list. Print the list to show that its order has changed.
- Use `reverse()` to change the order of your list again. Print the list to show it's back to its original order.
- Use `sort()` to change your list so it's stored in alphabetical order. Print the list to show that its order has been changed.
- Use `sort()` to change your list so it's stored in reverse alphabetical order. Print the list to show that its order has changed.

3-9. Dinner Guests: Working with one of the programs from Exercises 3-4 through 3-7 (page 46), use `len()` to print a message indicating the number of people you are inviting to dinner.

3-10. Every Function: Think of something you could store in a list. For example, you could make a list of mountains, rivers, countries, cities, languages, or anything else you'd like. Write a program that creates a list containing these items and then uses each function introduced in this chapter at least once.

Avoiding Index Errors When Working with Lists

One type of error is common to see when you're working with lists for the first time. Let's say you have a list with three items, and you ask for the fourth item:

```
motorcycles = ['honda', 'yamaha', 'suzuki']  
print(motorcycles[3])
```

This example results in an *index error*:

```
Traceback (most recent call last):
  File "motorcycles.py", line 3, in <module>
    print(motorcycles[3])
IndexError: list index out of range
```

Python attempts to give you the item at index 3. But when it searches the list, no item in `motorcycles` has an index of 3. Because of the off-by-one nature of indexing in lists, this error is typical. People think the third item is item number 3, because they start counting at 1. But in Python the third item is number 2, because it starts indexing at 0.

An index error means Python can't figure out the index you requested. If an index error occurs in your program, try adjusting the index you're asking for by one. Then run the program again to see if the results are correct.

Keep in mind that whenever you want to access the last item in a list you use the index `-1`. This will always work, even if your list has changed size since the last time you accessed it:

```
motorcycles = ['honda', 'yamaha', 'suzuki']
print(motorcycles[-1])
```

The index `-1` always returns the last item in a list, in this case the value `'suzuki'`:

```
'suzuki'
```

The only time this approach will cause an error is when you request the last item from an empty list:

```
motorcycles = []
print(motorcycles[-1])
```

No items are in `motorcycles`, so Python returns another index error:

```
Traceback (most recent call last):
  File "motorcycles.py", line 3, in <module>
    print(motorcycles[-1])
IndexError: list index out of range
```

NOTE

If an index error occurs and you can't figure out how to resolve it, try printing your list or just printing the length of your list. Your list might look much different than you thought it did, especially if it has been managed dynamically by your program. Seeing the actual list, or the exact number of items in your list, can help you sort out such logical errors.

TRY IT YOURSELF

3-11. Intentional Error: If you haven't received an index error in one of your programs yet, try to make one happen. Change an index in one of your programs to produce an index error. Make sure you correct the error before closing the program.

Summary

In this chapter you learned what lists are and how to work with the individual items in a list. You learned how to define a list and how to add and remove elements. You learned to sort lists permanently and temporarily for display purposes. You also learned how to find the length of a list and how to avoid index errors when you're working with lists.

In Chapter 4 you'll learn how to work with items in a list more efficiently. By looping through each item in a list using just a few lines of code you'll be able to work efficiently, even when your list contains thousands or millions of items.

4

WORKING WITH LISTS



In Chapter 3 you learned how to make a simple list, and you learned to work with the individual elements in a list. In this chapter you'll learn how to *loop* through an entire list using just a few lines of code regardless of how long the list is. Looping allows you to take the same action, or set of actions, with every item in a list. As a result, you'll be able to work efficiently with lists of any length, including those with thousands or even millions of items.

Looping Through an Entire List

You'll often want to run through all entries in a list, performing the same task with each item. For example, in a game you might want to move every element on the screen by the same amount, or in a list of numbers you might want to perform the same statistical operation on every element. Or perhaps you'll want to display each headline from a list of articles on a website. When you want to do the same action with every item in a list, you can use Python's `for` loop.

Let's say we have a list of magicians' names, and we want to print out each name in the list. We could do this by retrieving each name from the list individually, but this approach could cause several problems. For one, it would be repetitive to do this with a long list of names. Also, we'd have to change our code each time the list's length changed. A `for` loop avoids both of these issues by letting Python manage these issues internally.

Let's use a `for` loop to print out each name in a list of magicians:

```
magicians.py ❶ magicians = ['alice', 'david', 'carolina']  
              ❷ for magician in magicians:  
              ❸     print(magician)
```

We begin by defining a list at ❶, just as we did in Chapter 3. At ❷, we define a `for` loop. This line tells Python to pull a name from the list `magicians`, and store it in the variable `magician`. At ❸ we tell Python to print the name that was just stored in `magician`. Python then repeats lines ❷ and ❸, once for each name in the list. It might help to read this code as “For every magician in the list of magicians, print the magician's name.” The output is a simple printout of each name in the list:

```
alice  
david  
carolina
```

A Closer Look at Looping

The concept of looping is important because it's one of the most common ways a computer automates repetitive tasks. For example, in a simple loop like we used in *magicians.py*, Python initially reads the first line of the loop:

```
for magician in magicians:
```

This line tells Python to retrieve the first value from the list `magicians` and store it in the variable `magician`. This first value is `'alice'`. Python then reads the next line:

```
    print(magician)
```

Python prints the current value of `magician`, which is still `'alice'`. Because the list contains more values, Python returns to the first line of the loop:

```
for magician in magicians:
```

Python retrieves the next name in the list, `'david'`, and stores that value in `magician`. Python then executes the line:

```
    print(magician)
```

Python prints the current value of `magician` again, which is now `'david'`. Python repeats the entire loop once more with the last value in the list, `'carolina'`. Because no more values are in the list, Python moves on to the next line in the program. In this case nothing comes after the `for` loop, so the program simply ends.

When you're using loops for the first time, keep in mind that the set of steps is repeated once for each item in the list, no matter how many items are in the list. If you have a million items in your list, Python repeats these steps a million times—and usually very quickly.

Also keep in mind when writing your own `for` loops that you can choose any name you want for the temporary variable that holds each value in the list. However, it's helpful to choose a meaningful name that represents a single item from the list. For example, here's a good way to start a `for` loop for a list of cats, a list of dogs, and a general list of items:

```
for cat in cats:
for dog in dogs:
for item in list_of_items:
```

These naming conventions can help you follow the action being done on each item within a `for` loop. Using singular and plural names can help you identify whether a section of code is working with a single element from the list or the entire list.

Doing More Work Within a for Loop

You can do just about anything with each item in a `for` loop. Let's build on the previous example by printing a message to each magician, telling them that they performed a great trick:

```
magicians = ['alice', 'david', 'carolina']
for magician in magicians:
❶ print(magician.title() + ", that was a great trick!")
```

The only difference in this code is at ❶ where we compose a message to each magician, starting with that magician's name. The first time through the loop the value of `magician` is `'alice'`, so Python starts the first message with the name `'Alice'`. The second time through the message will begin with `'David'`, and the third time through the message will begin with `'Carolina'`.

The output shows a personalized message for each magician in the list:

```
Alice, that was a great trick!
David, that was a great trick!
Carolina, that was a great trick!
```

You can also write as many lines of code as you like in the `for` loop. Every indented line following the line `for magician in magicians` is considered *inside the loop*, and each indented line is executed once for each

value in the list. Therefore, you can do as much work as you like with each value in the list.

Let's add a second line to our message, telling each magician that we're looking forward to their next trick:

```
magicians = ['alice', 'david', 'carolina']
for magician in magicians:
    print(magician.title() + ", that was a great trick!")
❶    print("I can't wait to see your next trick, " + magician.title() + ".\n")
```

Because we have indented both print statements, each line will be executed once for every magician in the list. The newline ("\n") in the second print statement ❶ inserts a blank line after each pass through the loop. This creates a set of messages that are neatly grouped for each person in the list:

```
Alice, that was a great trick!
I can't wait to see your next trick, Alice.

David, that was a great trick!
I can't wait to see your next trick, David.

Carolina, that was a great trick!
I can't wait to see your next trick, Carolina.
```

You can use as many lines as you like in your for loops. In practice you'll often find it useful to do a number of different operations with each item in a list when you use a for loop.

Doing Something After a for Loop

What happens once a for loop has finished executing? Usually, you'll want to summarize a block of output or move on to other work that your program must accomplish.

Any lines of code after the for loop that are not indented are executed once without repetition. Let's write a thank you to the group of magicians as a whole, thanking them for putting on an excellent show. To display this group message after all of the individual messages have been printed, we place the thank you message after the for loop without indentation:

```
magicians = ['alice', 'david', 'carolina']
for magician in magicians:
    print(magician.title() + ", that was a great trick!")
    print("I can't wait to see your next trick, " + magician.title() + ".\n")
❶ print("Thank you, everyone. That was a great magic show!")
```

The first two print statements are repeated once for each magician in the list, as you saw earlier. However, because the line at ❶ is not indented, it's printed only once:

```
Alice, that was a great trick!
I can't wait to see your next trick, Alice.

David, that was a great trick!
I can't wait to see your next trick, David.

Carolina, that was a great trick!
I can't wait to see your next trick, Carolina.

Thank you, everyone. That was a great magic show!
```

When you're processing data using a `for` loop, you'll find that this is a good way to summarize an operation that was performed on an entire data set. For example, you might use a `for` loop to initialize a game by running through a list of characters and displaying each character on the screen. You might then write an unindented block after this loop that displays a Play Now button after all the characters have been drawn to the screen.

Avoiding Indentation Errors

Python uses indentation to determine when one line of code is connected to the line above it. In the previous examples, the lines that printed messages to individual magicians were part of the `for` loop because they were indented. Python's use of indentation makes code very easy to read. Basically, it uses whitespace to force you to write neatly formatted code with a clear visual structure. In longer Python programs, you'll notice blocks of code indented at a few different levels. These indentation levels help you gain a general sense of the overall program's organization.

As you begin to write code that relies on proper indentation, you'll need to watch for a few common *indentation errors*. For example, people sometimes indent blocks of code that don't need to be indented or forget to indent blocks that need to be indented. Seeing examples of these errors now will help you avoid them in the future and correct them when they do appear in your own programs.

Let's examine some of the more common indentation errors.

Forgetting to Indent

Always indent the line after the `for` statement in a loop. If you forget, Python will remind you:

```
magicians.py    magicians = ['alice', 'david', 'carolina']
                for magician in magicians:
❶ print(magician)
```

The print statement at ❶ should be indented, but it's not. When Python expects an indented block and doesn't find one, it lets you know which line it had a problem with.

```
File "magicians.py", line 3
    print(magician)
    ^
IndentationError: expected an indented block
```

You can usually resolve this kind of indentation error by indenting the line or lines immediately after the for statement.

Forgetting to Indent Additional Lines

Sometimes your loop will run without any errors but won't produce the expected result. This can happen when you're trying to do several tasks in a loop and you forget to indent some of its lines.

For example, this is what happens when we forget to indent the second line in the loop that tells each magician we're looking forward to their next trick:

```
magicians = ['alice', 'david', 'carolina']
for magician in magicians:
    print(magician.title() + ", that was a great trick!")
❶ print("I can't wait to see your next trick, " + magician.title() + ".\n")
```

The print statement at ❶ is supposed to be indented, but because Python finds at least one indented line after the for statement, it doesn't report an error. As a result, the first print statement is executed once for each name in the list because it is indented. The second print statement is not indented, so it is executed only once after the loop has finished running. Because the final value of magician is 'carolina', she is the only one who receives the "looking forward to the next trick" message:

```
Alice, that was a great trick!
David, that was a great trick!
Carolina, that was a great trick!
I can't wait to see your next trick, Carolina.
```

This is a *logical error*. The syntax is valid Python code, but the code does not produce the desired result because a problem occurs in its logic. If you expect to see a certain action repeated once for each item in a list and it's executed only once, determine whether you need to simply indent a line or a group of lines.

Indenting Unnecessarily

If you accidentally indent a line that doesn't need to be indented, Python informs you about the unexpected indent:

hello_world.py

```
❶ message = "Hello Python world!"  
    print(message)
```

We don't need to indent the `print` statement at ❶, because it doesn't *belong* to the line above it; hence, Python reports that error:

```
File "hello_world.py", line 2  
    print(message)  
    ^
```

IndentationError: unexpected indent

You can avoid unexpected indentation errors by indenting only when you have a specific reason to do so. In the programs you're writing at this point, the only lines you should indent are the actions you want to repeat for each item in a `for` loop.

Indenting Unnecessarily After the Loop

If you accidentally indent code that should run after a loop has finished, that code will be repeated once for each item in the list. Sometimes this prompts Python to report an error, but often you'll receive a simple logical error.

For example, let's see what happens when we accidentally indent the line that thanked the magicians as a group for putting on a good show:

```
magicians = ['alice', 'david', 'carolina']  
for magician in magicians:  
    print(magician.title() + ", that was a great trick!")  
    print("I can't wait to see your next trick, " + magician.title() + ".\n")  
❶ print("Thank you everyone, that was a great magic show!")
```

Because the line at ❶ is indented, it's printed once for each person in the list, as you can see at ❷:

```
Alice, that was a great trick!  
I can't wait to see your next trick, Alice.
```

```
❷ Thank you everyone, that was a great magic show!  
David, that was a great trick!  
I can't wait to see your next trick, David.
```

```
❷ Thank you everyone, that was a great magic show!  
Carolina, that was a great trick!  
I can't wait to see your next trick, Carolina.
```

```
❷ Thank you everyone, that was a great magic show!
```

This is another logical error, similar to the one in “Forgetting to Indent Additional Lines” on page 58. Because Python doesn’t know what you’re trying to accomplish with your code, it will run all code that is written in valid syntax. If an action is repeated many times when it should be executed only once, determine whether you just need to unindent the code for that action.

Forgetting the Colon

The colon at the end of a for statement tells Python to interpret the next line as the start of a loop.

```
magicians = ['alice', 'david', 'carolina']  
❶ for magician in magicians  
    print(magician)
```

If you accidentally forget the colon, as shown at ❶, you’ll get a syntax error because Python doesn’t know what you’re trying to do. Although this is an easy error to fix, it’s not always an easy error to find. You’d be surprised by the amount of time programmers spend hunting down single-character errors like this. Such errors are difficult to find because we often just see what we expect to see.

TRY IT YOURSELF

4-1. Pizzas: Think of at least three kinds of your favorite pizza. Store these pizza names in a list, and then use a for loop to print the name of each pizza.

- Modify your for loop to print a sentence using the name of the pizza instead of printing just the name of the pizza. For each pizza you should have one line of output containing a simple statement like *I like pepperoni pizza*.
- Add a line at the end of your program, outside the for loop, that states how much you like pizza. The output should consist of three or more lines about the kinds of pizza you like and then an additional sentence, such as *I really love pizza!*

4-2. Animals: Think of at least three different animals that have a common characteristic. Store the names of these animals in a list, and then use a for loop to print out the name of each animal.

- Modify your program to print a statement about each animal, such as *A dog would make a great pet*.
- Add a line at the end of your program stating what these animals have in common. You could print a sentence such as *Any of these animals would make a great pet!*

Making Numerical Lists

Many reasons exist to store a set of numbers. For example, you'll need to keep track of the positions of each character in a game, and you might want to keep track of a player's high scores as well. In data visualizations, you'll almost always work with sets of numbers, such as temperatures, distances, population sizes, or latitude and longitude values, among other types of numerical sets.

Lists are ideal for storing sets of numbers, and Python provides a number of tools to help you work efficiently with lists of numbers. Once you understand how to use these tools effectively, your code will work well even when your lists contain millions of items.

Using the range() Function

Python's `range()` function makes it easy to generate a series of numbers. For example, you can use the `range()` function to print a series of numbers like this:

```
numbers.py for value in range(1,5):  
           print(value)
```

Although this code looks like it should print the numbers from 1 to 5, it doesn't print the number 5:

```
1  
2  
3  
4
```

In this example, `range()` prints only the numbers 1 through 4. This is another result of the off-by-one behavior you'll see often in programming languages. The `range()` function causes Python to start counting at the first value you give it, and it stops when it reaches the second value you provide. Because it stops at that second value, the output never contains the end value, which would have been 5 in this case.

To print the numbers from 1 to 5, you would use `range(1,6)`:

```
for value in range(1,6):  
    print(value)
```

This time the output starts at 1 and ends at 5:

```
1  
2  
3  
4  
5
```

If your output is different than what you expect when you're using `range()`, try adjusting your end value by 1.

Using range() to Make a List of Numbers

If you want to make a list of numbers, you can convert the results of `range()` directly into a list using the `list()` function. When you wrap `list()` around a call to the `range()` function, the output will be a list of numbers.

In the example in the previous section, we simply printed out a series of numbers. We can use `list()` to convert that same set of numbers into a list:

```
numbers = list(range(1,6))
print(numbers)
```

And this is the result:

```
[1, 2, 3, 4, 5]
```

We can also use the `range()` function to tell Python to skip numbers in a given range. For example, here's how we would list the even numbers between 1 and 10:

```
even_numbers.py even_numbers = list(range(2,11,2))
print(even_numbers)
```

In this example, the `range()` function starts with the value 2 and then adds 2 to that value. It adds 2 repeatedly until it reaches or passes the end value, 11, and produces this result:

```
[2, 4, 6, 8, 10]
```

You can create almost any set of numbers you want to using the `range()` function. For example, consider how you might make a list of the first 10 square numbers (that is, the square of each integer from 1 through 10). In Python, two asterisks (`**`) represent exponents. Here's how you might put the first 10 square numbers into a list:

```
squares.py ❶ squares = []
            ❷ for value in range(1,11):
            ❸     square = value**2
            ❹     squares.append(square)

            ❺ print(squares)
```

We start with an empty list called `squares` at ❶. At ❷, we tell Python to loop through each value from 1 to 10 using the `range()` function. Inside the loop, the current value is raised to the second power and stored in the

variable `square` at ❸. At ❹, each new value of `square` is appended to the list `squares`. Finally, when the loop has finished running, the list of squares is printed at ❺:

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

To write this code more concisely, omit the temporary variable `square` and append each new value directly to the list:

```
squares = []  
for value in range(1,11):  
❶    squares.append(value**2)  
  
print(squares)
```

The code at ❶ does the same work as the lines at ❸ and ❹ in `squares.py`. Each value in the loop is raised to the second power and then immediately appended to the list of squares.

You can use either of these two approaches when you're making more complex lists. Sometimes using a temporary variable makes your code easier to read; other times it makes the code unnecessarily long. Focus first on writing code that you understand clearly, which does what you want it to do. Then look for more efficient approaches as you review your code.

Simple Statistics with a List of Numbers

A few Python functions are specific to lists of numbers. For example, you can easily find the minimum, maximum, and sum of a list of numbers:

```
>>> digits = [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]  
>>> min(digits)  
0  
>>> max(digits)  
9  
>>> sum(digits)  
45
```

NOTE

The examples in this section use short lists of numbers in order to fit easily on the page. They would work just as well if your list contained a million or more numbers.

List Comprehensions

The approach described earlier for generating the list `squares` consisted of using three or four lines of code. A *list comprehension* allows you to generate this same list in just one line of code. A list comprehension combines the for loop and the creation of new elements into one line, and automatically appends each new element. List comprehensions are not always presented to beginners, but I have included them here because you'll most likely see them as soon as you start looking at other people's code.

The following example builds the same list of square numbers you saw earlier but uses a list comprehension:

```
squares.py squares = [value**2 for value in range(1,11)]
print(squares)
```

To use this syntax, begin with a descriptive name for the list, such as `squares`. Next, open a set of square brackets and define the expression for the values you want to store in the new list. In this example the expression is `value**2`, which raises the value to the second power. Then, write a `for` loop to generate the numbers you want to feed into the expression, and close the square brackets. The `for` loop in this example is `for value in range(1,11)`, which feeds the values 1 through 10 into the expression `value**2`. Notice that no colon is used at the end of the `for` statement.

The result is the same list of square numbers you saw earlier:

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

It takes practice to write your own list comprehensions, but you'll find them worthwhile once you become comfortable creating ordinary lists. When you're writing three or four lines of code to generate lists and it begins to feel repetitive, consider writing your own list comprehensions.

TRY IT YOURSELF

4-3. Counting to Twenty: Use a `for` loop to print the numbers from 1 to 20, inclusive.

4-4. One Million: Make a list of the numbers from one to one million, and then use a `for` loop to print the numbers. (If the output is taking too long, stop it by pressing `CTRL-C` or by closing the output window.)

4-5. Summing a Million: Make a list of the numbers from one to one million, and then use `min()` and `max()` to make sure your list actually starts at one and ends at one million. Also, use the `sum()` function to see how quickly Python can add a million numbers.

4-6. Odd Numbers: Use the third argument of the `range()` function to make a list of the odd numbers from 1 to 20. Use a `for` loop to print each number.

4-7. Threes: Make a list of the multiples of 3 from 3 to 30. Use a `for` loop to print the numbers in your list.

4-8. Cubes: A number raised to the third power is called a *cube*. For example, the cube of 2 is written as `2**3` in Python. Make a list of the first 10 cubes (that is, the cube of each integer from 1 through 10), and use a `for` loop to print out the value of each cube.

4-9. Cube Comprehension: Use a list comprehension to generate a list of the first 10 cubes.

Working with Part of a List

In Chapter 3 you learned how to access single elements in a list, and in this chapter you've been learning how to work through all the elements in a list. You can also work with a specific group of items in a list, which Python calls a *slice*.

Slicing a List

To make a slice, you specify the index of the first and last elements you want to work with. As with the `range()` function, Python stops one item before the second index you specify. To output the first three elements in a list, you would request indices 0 through 3, which would return elements 0, 1, and 2.

The following example involves a list of players on a team:

```
players.py  players = ['charles', 'martina', 'michael', 'florence', 'eli']  
❶ print(players[0:3])
```

The code at ❶ prints a slice of this list, which includes just the first three players. The output retains the structure of the list and includes the first three players in the list:

```
['charles', 'martina', 'michael']
```

You can generate any subset of a list. For example, if you want the second, third, and fourth items in a list, you would start the slice at index 1 and end at index 4:

```
players = ['charles', 'martina', 'michael', 'florence', 'eli']  
print(players[1:4])
```

This time the slice starts with 'martina' and ends with 'florence':

```
['martina', 'michael', 'florence']
```

If you omit the first index in a slice, Python automatically starts your slice at the beginning of the list:

```
players = ['charles', 'martina', 'michael', 'florence', 'eli']  
print(players[:4])
```

Without a starting index, Python starts at the beginning of the list:

```
['charles', 'martina', 'michael', 'florence']
```

A similar syntax works if you want a slice that includes the end of a list. For example, if you want all items from the third item through the last item, you can start with index 2 and omit the second index:

```
players = ['charles', 'martina', 'michael', 'florence', 'eli']
print(players[2:])
```

Python returns all items from the third item through the end of the list:

```
['michael', 'florence', 'eli']
```

This syntax allows you to output all of the elements from any point in your list to the end regardless of the length of the list. Recall that a negative index returns an element a certain distance from the end of a list; therefore, you can output any slice from the end of a list. For example, if we want to output the last three players on the roster, we can use the slice `players[-3:]`:

```
players = ['charles', 'martina', 'michael', 'florence', 'eli']
print(players[-3:])
```

This prints the names of the last three players and would continue to work as the list of players changes in size.

Looping Through a Slice

You can use a slice in a for loop if you want to loop through a subset of the elements in a list. In the next example we loop through the first three players and print their names as part of a simple roster:

```
players = ['charles', 'martina', 'michael', 'florence', 'eli']

print("Here are the first three players on my team:")
❶ for player in players[:3]:
    print(player.title())
```

Instead of looping through the entire list of players at ❶, Python loops through only the first three names:

```
Here are the first three players on my team:
Charles
Martina
Michael
```

Slices are very useful in a number of situations. For instance, when you're creating a game, you could add a player's final score to a list every time that player finishes playing. You could then get a player's top three scores by sorting the list in decreasing order and taking a slice that includes just the first three scores. When you're working with data, you can use slices to process

your data in chunks of a specific size. Or, when you're building a web application, you could use slices to display information in a series of pages with an appropriate amount of information on each page.

Copying a List

Often, you'll want to start with an existing list and make an entirely new list based on the first one. Let's explore how copying a list works and examine one situation in which copying a list is useful.

To copy a list, you can make a slice that includes the entire original list by omitting the first index and the second index (`[:]`). This tells Python to make a slice that starts at the first item and ends with the last item, producing a copy of the entire list.

For example, imagine we have a list of our favorite foods and want to make a separate list of foods that a friend likes. This friend likes everything in our list so far, so we can create their list by copying ours:

```
foods.py ❶ my_foods = ['pizza', 'falafel', 'carrot cake']
          ❷ friend_foods = my_foods[:]

print("My favorite foods are:")
print(my_foods)

print("\nMy friend's favorite foods are:")
print(friend_foods)
```

At ❶ we make a list of the foods we like called `my_foods`. At ❷ we make a new list called `friend_foods`. We make a copy of `my_foods` by asking for a slice of `my_foods` without specifying any indices and store the copy in `friend_foods`. When we print each list, we see that they both contain the same foods:

```
My favorite foods are:
['pizza', 'falafel', 'carrot cake']

My friend's favorite foods are:
['pizza', 'falafel', 'carrot cake']
```

To prove that we actually have two separate lists, we'll add a new food to each list and show that each list keeps track of the appropriate person's favorite foods:

```
my_foods = ['pizza', 'falafel', 'carrot cake']
❶ friend_foods = my_foods[:]

❷ my_foods.append('cannoli')
❸ friend_foods.append('ice cream')

print("My favorite foods are:")
print(my_foods)
```
